CS229: Machine Learning — 系统学习笔记（数学直觉 × 算法伪代码）

> **资料来源**：斯坦福大学 CS229 "Machine Learning" 官方网站（https://cs229.stanford.edu/） **笔记定位**：基于公开 Syllabus 与公开课程讲义整理的系统学习笔记，重点解释数学公式的直觉、给出核心算法的伪代码与逻辑解说。 **生成日期**：2026-09-01

---

0. 数据获取与可访问性记录（Phase 1 交付）

0.1 访问过程摘要

资源	URL	可访问性	说明
课程主页	https://cs229.stanford.edu/	✅ 公开	课程描述、讲师信息、课程信息、日程表（Summer 2026 版，含 20 讲）
Course Logistics & FAQ	Google Docs（主页 Quick Links）	✅ 公开	CS229 Summer 2026 完整 FAQ：先修、Honor Code、评分、补交、AI 工具政策等
Syllabus（Fall 2021）	https://cs229.stanford.edu/syllabus-fall2021.html	✅ 公开	含完整讲座日程表与讲义链接
Syllabus（Fall 2020 / Spring 2021）	https://cs229.stanford.edu/syllabus-fall2020.html 等	✅ 公开	含完整日程表
讲义（Lecture Notes）	https://cs229.stanford.edu/notes2021fall/*.pdf	✅ 公开（本季度主页声明“仅斯坦福成员”后实际仍公开于旧目录）	cs229-notes1~12、deep_learning_notes、ML-advice 等
Lecture Notes（2026 当季）	主页 "Syllabus and Course Materials"	🔒 受限	需 Stanford 邮箱登录 Google Drive 查看
Problem Sets / 作业	Ed / Gradescope	🔒 受限	FAQ 明确说明作业仅通过 Ed 发布，网站与 Canvas 均不公开

资源	URL	可访问性	说明
Canvas	https://canvas.stanford.edu/courses/228316	🔒 受限	需 Stanford 账号；录播、办公时间日历、成绩
Ed 论坛	https://edstem.org/us/courses/99528	🔒 受限	课程唯一官方沟通渠道
期末考试	线下（Summer 2026: 2026-08-15 19:00–22:00）	🔒 受限	需在校参加；SCPD/CGOE 学生远程监考

0.2 未公开资源记录

按主页说明，以下资源**仅对斯坦福大学附属人员开放**，本笔记无法获取其内容，仅记录其存在：

- **Problem Set 0–4**（作业仅通过 Ed 发布，不公开）
- **Lecture 录播视频**（仅 Canvas）
- **Ed 论坛讨论与官方答疑**
- **当季（2026）讲义 PDF**（Google Drive 分享链接需 Stanford 登录）
- **Midterm / Final Exam 试卷与答案**
- **最终项目报告与 Poster**

> ⚠️ 提示：主页 Quick Links 明确写道 "All links will require you to be logged into your Stanford email to access. Course documents are only shared with Stanford University affiliates." 本笔记的所有内容均来自公开渠道（主页、公开 Syllabus、公开讲义 PDF、公开 FAQ）。

---

1. 课程概览（Course Overview）

1.1 课程定位

CS229 是斯坦福大学计算机科学系的研究生/高年级本科课程，提供对**机器学习与统计模式识别**的广泛而深入的理论介绍。它不只是一门“调库”课：课程从概率与线性代数的第一性原理出发，推导每个算法背后的数学，并训练学生用 Python/NumPy 从零实现核心算法。

1.2 课程范围（官方 Course Description）

> This course provides a broad introduction to machine learning and statistical pattern recognition. Topics include: supervised learning (generative learning, parametric/non-parametric learning, neural networks); unsupervised learning (clustering, dimensionality reduction); learning theory (bias/variance tradeoffs, practical advice); reinforcement learning and adaptive control. The course will also discuss recent applications of machine learning, such as to robotic control, data mining, autonomous navigation, bioinformatics, speech recognition, and text and web data processing.

1.3 四大主题模块

模块	覆盖讲座	核心问题	代表算法
监督学习 Supervised Learning	L2–L7, L9–L12	给定带标签数据 (x, y)，学习映射 $x \to y$	线性回归、逻辑回归、GLM、GDA、朴素贝叶斯、SVM、决策树、Boosting、神经网络
无监督学习 Unsupervised Learning	L8, L16	给定无标签数据，发现隐藏结构	K-Means、GMM、EM、PCA、ICA
学习理论 Learning Theory	L5, L13	模型为何泛化？偏差/方差如何权衡？如何调试 ML 系统？	正则化、模型选择、交叉验证、误差分析、Ablation
强化学习 Reinforcement Learning	L14–L15	智能体如何通过试错（奖励信号）学习序贯决策？	MDP、值迭代、策略迭代、Q-learning、REINFORCE
现代主题 Modern Topics	L17–L20	大规模语言模型、公平性、可解释性、隐私	Transformer、RAG、微调、prompt 优化

1.4 教学目标

1. **理论深度**：能读懂并推导 $J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$ 这类公式，理解其来源（概率解释、MLE）。
2. **算法直觉**：知道每个算法**在做什么**、**为什么这样做**、**什么时候适用**。
3. **工程能力**：用 Python/NumPy 实现算法、调试学习算法、做误差分析与消融实验。

---

2. 课程信息与政策速览（来自主页 + 公开 FAQ）

2.1 课程基本信息（Summer 2026）

- **讲师**：Jehangir Amjad、Anand Avati（当季）；经典版本讲师为 Andrew Ng、Tengyu Ma 等。

- **时间地点**：周二/周四 16:30–18:15，NVIDIA Auditorium；周五 CA Lecture 13:30–15:00（Skilling Auditorium）。
- **先修要求**：
- 计算机基础与 Python/NumPy 编程能力（CS106A/CS106B 等效）；
- 概率论（CS109 / MATH151 / STATS116 等效）；
- 多元微积分与线性代数（MATH51 / CS205L 等效）。
- **无指定教材**：讲义（Notes）是主要学习材料，公开于 Syllabus 页面。

2.2 评分与政策（来自公开 FAQ）

- **评分构成（Summer 2026）**：3 个作业共 50% + 期末考试 50%（期末必须及格才能通过课程）。
- **Late Policy**：每人共 3 个免费迟到日；之后每天扣 20%；超过 3 天不接受。
- **Honor Code**：允许讨论，但必须独立撰写答案；禁止参考往年答案；禁止公开张贴作业解。
- **AI 工具政策**：可将生成式 AI 视为“人类合作者”——不得直接索取答案或复制解法，使用需注明。
- **Ed 是唯一官方沟通渠道**；禁止直接私信讲师。
- **3 单位 vs 4 单位**：工作量相同，可自由选择。

---

3. 课程日程表（Summer 2026，来自主页）

> 注：主页日程表日期标注存在模板残留（Lecture 1 为 2026-01-05，其后为 2025 年日期），本笔记以**讲座主题顺序**为准。

讲次	主题	说明/关联作业
L1	Introduction	PS0 发布
L2	Supervised learning setup. LMS	PS1 发布
L3	Weighted Least Squares. Logistic regression. Newton's Method	
L4	Dataset split; Exponential family. Generalized Linear Models	PS0 截止（不计分）
L5	Bias-variance tradeoff, regularization	Final Project Proposal 截止
L6	Gaussian discriminant analysis. Naive Bayes, Laplace Smoothing	PS2 发布；PS1 截止
L7	Kernels. SVM	
L8	K-Means. GMM. Expectation Maximization	
L9	Decision trees	
L10	Boosting	PS3 发布；PS2 截止

讲次	主题	说明/关联作业
L11	Neural Networks 1	
L12	Neural Networks 2 (backprop)	
—	**MIDTERM**（第 6 周）	3 小时笔试
L13	ML Advice	
L14	Basic concepts in RL, value iteration, policy iteration	PS4 发布；PS3 截止
L15	Model-based RL, value function approximator	
L16	PCA	Final Project Milestone 截止
L17	LLMs — learning tasks, language modeling, embeddings, transformers	
L18	LLMs — RAG, fine-tuning, prompt optimization, safety	PS4 截止
L19	Fairness, algorithmic bias, explainability, privacy	
L20	Fairness, algorithmic bias, explainability, privacy	
—	Final Project Report / Poster Session	

4. 按讲次的数据记录（Phase 1 结构化产物）


```
[
  {"lecture_number": 1,  "topic": "Introduction", "key_concepts_raw": ["machine learning la
  {"lecture_number": 2,  "topic": "Supervised learning setup. LMS (Linear Regression)", "ke
  {"lecture_number": 3,  "topic": "Weighted Least Squares. Logistic regression. Newton's Me
  {"lecture_number": 4,  "topic": "Dataset split; Exponential family. Generalized Linear Mo
  {"lecture_number": 5,  "topic": "Bias-variance tradeoff, regularization", "key_concepts_r
  {"lecture_number": 6,  "topic": "Gaussian discriminant analysis. Naive Bayes, Laplace Smo
  {"lecture_number": 7,  "topic": "Kernels. SVM", "key_concepts_raw": ["feature mapping", "
  {"lecture_number": 8,  "topic": "K-Means. GMM. Expectation Maximization", "key_concepts_r
  {"lecture_number": 9,  "topic": "Decision trees", "key_concepts_raw": ["decision trees",
  {"lecture_number": 10, "topic": "Boosting", "key_concepts_raw": ["ensemble learning", "Ad
  {"lecture_number": 11, "topic": "Neural Networks 1", "key_concepts_raw": ["neural network
  {"lecture_number": 12, "topic": "Neural Networks 2 (backprop)", "key_concepts_raw": ["bac
  {"lecture_number": 13, "topic": "ML Advice", "key_concepts_raw": ["debugging learning alg
  {"lecture_number": 14, "topic": "Basic concepts in RL, value iteration, policy iteration"
  {"lecture_number": 15, "topic": "Model-based RL, value function approximator", "key_conce
  {"lecture_number": 16, "topic": "PCA", "key_concepts_raw": ["dimensionality reduction", "
  {"lecture_number": 17, "topic": "Large language models — learning tasks, language modelin
  {"lecture_number": 18, "topic": "Large language models — RAG, fine-tuning, prompt optimiz
  {"lecture_number": 19, "topic": "Fairness, algorithmic bias, explainability, privacy", "k
  {"lecture_number": 20, "topic": "Fairness, algorithmic bias, explainability, privacy", "k
]
```

第一部分：监督学习（Supervised Learning）

Lecture 1: Introduction

概述

本讲是课程的开篇，回答三个问题：**机器学习是什么**（从数据中自动发现规律）、**学什么**（三大学习范式：监督、无监督、强化学习）、**为什么现在学**（数据与算力的爆发）。同时介绍课程的结构、工具（Python/NumPy）与评分方式，为后续所有讲座建立共同语言。

核心概念与数学直觉

- **机器学习的形式化定义**：一个程序被认为“在学习”，如果它在任务 T 上的性能度量 P 随着经验 E 的增加而提高。例如：垃圾邮件过滤中，T=判断邮件是否为垃圾，P=分类准确率，E=已标注的邮件样本。
- **监督学习 (Supervised Learning)**：训练数据为输入–输出对 $\{(x^{(i)}, y^{(i)})\}_{i=1}^{m}$，目标是学习映射 $h : \mathcal{X} \to \mathcal{Y}$（称为**假设函数 hypothesis**）。
  - y 连续 → **回归 (Regression)**（如房价预测）。
  - y 离散 → **分类 (Classification)**（如垃圾邮件判断）。
  - *直观解释*：像学生做“带标准答案的习题集”——每道题 $(x^{(i)})$ 都有正确答案 $y^{(i)}$，学完后要能回答没见过的题目。
- **无监督学习 (Unsupervised Learning)**：训练数据只有输入 $\{x^{(i)}\}$，没有标签。
  - 目标是发现数据内在结构：**聚类**（K-Means、GMM）、**降维**（PCA、ICA）。
  - *直观解释*：像整理一堆没有标签的照片——自动按人脸、场景分组（聚类），或找出最能区分照片的主要维度（降维）。
- **强化学习 (Reinforcement Learning)**：没有标签，只有**奖励信号 (reward)**。智能体 (agent) 与环境 (environment) 交互，通过试错最大化累积奖励。
  - *直观解释*：像训练小狗——不告诉它“先抬左腿再抬右腿”，只在它做得对时给零食（正奖励）、做错时不给（负奖励）。
- **学习范式的对比直觉**：
范式	数据	反馈形式	典型目标
监督学习	(x, y) 对	直接答案	拟合 $x \to y$
无监督学习	x	无	发现结构
强化学习	状态/动作序列	稀疏的奖励	最大化长期回报

算法伪代码与逻辑解说

本讲以概念为主，无核心算法。这里给出贯穿全课程的方法论伪代码——**机器学习项目的一般流程**：

伪代码：监督学习项目流程

```
输入:
    - 原始数据 D（特征 + 可能的标签）
    - 任务类型 T（回归/分类/聚类/...）

输出:
    - 训练好的模型 f，及其在测试集上的性能报告

1. 收集数据并清洗（处理缺失值、异常值）
2. 划分数据集: train / dev (validation) / test
3. 选择模型族（线性、树、神经网络...）与损失函数
4. 在训练集上拟合模型（优化损失）
5. 在 dev 集上评估，诊断问题:
    - 高偏差(欠拟合) → 增加特征/容量
    - 高方差(过拟合) → 正则化/更多数据
6. 调参并重复 4-5
7. 最终在 test 集上报告性能（只测一次！）
```

【算法逻辑解说】 1. **数据划分是关键纪律**：`test` 集必须像“未来数据”一样被隔离——如果反复用测试集调参，模型会“记住”测试集（数据泄漏），评估就失去意义。课程 L4 专门讲解数据集划分，L5 讲偏差/方差诊断。 2. **dev 集是“试衣间”**：所有调参、模型选择都在 dev 集上做；test 集只在最后使用一次。 3. **循环改进**：机器学习是迭代工程——训练 → 诊断 → 改进 → 再训练，L13（ML Advice）给出系统化的诊断方法。

关键要点

1. 机器学习 = 数据 + 模型 + 优化 + 评估；四大模块缺一不可。
2. 监督/无监督/强化学习的本质区别在于**反馈形式**（直接答案 / 无 / 稀疏奖励）。
3. 课程的理论主线：从**概率建模**（MLE）推导出**损失函数**，再用**优化算法**求解——L2–L4 会完整展示这条主线。
4. 工具栈：Python + NumPy（课程先修要求），用向量化实现算法。

常见误区与注意事项

- **混淆“训练误差低”与“模型好”**：模型可能在训练集上完美但泛化差（过拟合）。评估必须看未见数据。
- **过早优化细节**：先跑通一个简单基线（如线性模型），再逐步增加复杂度。
- **忽略数据质量**：垃圾进垃圾出 (garbage in, garbage out)——数据清洗与特征工程往往比换模型更有效。
- **误以为“深度学习=机器学习全部”**：CS229 强调理解所有经典算法（线性回归、SVM、EM...），它们是大模型的基础构件。

思考题

1. **问题**：判断以下场景属于哪种学习范式：(a) 根据用户历史点击预测其下一步点击的商品；(b) 将新闻自动聚类成主题；(c) 让机器人学习走路。
    - **答案**：(a) 监督学习（有用户–商品对作为标签）；(b) 无监督学习（无标签聚类）；(c) 强化学习（只有“前进/摔倒”的奖励信号）。
2. **问题**：为什么不能把测试集用于模型选择?

- **答案**：因为模型选择的本质也是“学习”——如果依据测试集表现选模型，测试集的信息就泄漏进了模型，最终报告的性能会系统性偏乐观（过拟合测试集）。必须用独立的 dev 集做选择，test 集只用于最终的一次性评估。

Lecture 2: Supervised Learning Setup. LMS（线性回归与最小均方）

概述

本讲正式建立监督学习的数学框架：定义**假设函数**、**代价函数**与两种求解算法——**梯度下降（批量/随机）** 与**正规方程**。核心思想：把“学习”转化为“最小化一个可微的代价函数”，这是整个课程最基础也最通用的一步。

核心概念与数学直觉

- **监督学习问题设定**：训练集 $\{(x^{(i)}, y^{(i)});\ i = 1, \ldots, m\}$，$x^{(i)} \in \mathbb{R}^n$（$n$ 个特征），$y^{(i)} \in \mathbb{R}$。目标：学习假设 h 使 $h(x) \approx y$。
- **假设函数 (Hypothesis)** `$h_\theta(x)$`：输入特征的线性组合。
  - *直观解释*：在二维平面中，$h_\theta(x) = \theta_0 + \theta_1 x$ 就是一条直线——我们要找一条“最贴近所有数据点”的直线。
  - *数学形式*：`$h_\theta(x) = \sum_{j=0}^{n} \theta_j x_j = \theta^T x$`（约定 $x_0 = 1$，把截距吸收进参数向量）。
    - $\theta \in \mathbb{R}^{n+1}$：**参数/权重**，决定直线的斜率与截距，是我们要学习的对象。
    - x_j：第 j 个特征；$x_0 = 1$ 是偏置项（bias term）。
    - $\theta^T x$：向量点积，把“每条特征对预测的贡献 $\theta_j x_j$”累加起来。
  - *直觉*：每个参数 θ_j 可理解为“特征 x_j 每增加一个单位，预测值变化多少”——这就是**可解释性**的来源（在特征独立、量纲相当时）。
- **代价函数 (Cost Function)** `$J(\theta)$`：衡量预测与真值的差距，即**均方误差 (MSE)**：

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

```
*   $m$：训练样本数。
*   $h_\theta(x^{(i)})$：第 $i$ 个样本的预测值。
*   $y^{(i)}$：第 $i$ 个样本的真实值。
*   $(h_\theta(x^{(i)}) - y^{(i)})^2$：**平方误差**——平方保证非负，且惩罚大误差远重于小误差（非线性放
*   $\frac{1}{2m}$：$1/m$ 是取平均（与样本量无关）；$1/2$ 纯粹为了方便—求导时平方的 2 与 $1/2$ 抵消，
*   *直觉*：代价函数像“卷尺”—测量拟合曲线与数据点的总偏差；目标是把总偏差压到最小。
```

- **为什么 MSE 是合理的？（概率解释，L2 后半部分）**：假设 $y^{(i)} = \theta^T x^{(i)} + \epsilon^{(i)}$，其中噪声 $\epsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$ 独立同分布。则由**最大似然估计 (MLE)**：

$$\ell(\theta) = \log \prod_{i=1}^{m} p(y^{(i)}|x^{(i)};\theta) = m \log \frac{1}{\sqrt{2\pi}\sigma} - \frac{1}{2\sigma^2} \sum_{i=1}^{m} (y^{(i)} - \theta^T x^{(i)})^2$$

```
最大化对数似然 $\ell(\theta)$ **等价于**最小化 $J(\theta)$! 这揭示了 MSE 的“出身”：它来自“高斯噪声 +
```

- **正规方程 (Normal Equation)**：代价函数 J 对 θ 求导置零，得到闭式解：

$$\nabla_\theta J(\theta) = \frac{1}{m} X^T (X\theta - y) = 0 \quad \Longrightarrow \quad \theta = (X^T X)^{-1} X^T y$$

```
*   $X \in \mathbb{R}^{m \times (n+1)}$: **设计矩阵**, 第 $i$ 行为样本 $x^{(i)}$ (含 $x_0=1$)。
*   $y \in \mathbb{R}^{m}$: 标签向量。
*   $X^T X$: 若可逆 (特征线性无关), 一步求出全局最优; 复杂度 $O(n^3)$ (求逆)。
```

算法伪代码与逻辑解说：批量梯度下降 (Batch Gradient Descent)

伪代码

```
输入:
    - 训练数据 (X, y), X 为 m×(n+1) 设计矩阵
    - 学习率 alpha (η)
    - 收敛阈值 epsilon, 最大迭代次数 max_iters

输出:
    - 最优参数 theta

1. 初始化 theta = 0 (或小随机值), iter = 0
2. 循环直到收敛:
    2.1 计算梯度: grad = (1/m) * X^T * (X*theta - y)   // 全量样本
    2.2 更新: theta = theta - alpha * grad
    2.3 iter = iter + 1
    2.4 若 ||grad|| < epsilon 或 iter >= max_iters: 终止
3. 返回 theta
```

【算法逻辑解说】 1. **梯度是什么**：$\nabla_\theta J$ 是一个向量，指向 J 在当前 θ 处**上升最快**的方向。减去它（乘学习率 α）就是沿**下降最快**的方向迈一步——这就是“下山”的比喻：闭着眼感受最陡的坡，每次沿最陡方向挪一步。 2. **更新规则的标量形式**（理解用）：对每个参数 θ_j：

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

注意 $(h_\theta(x^{(i)}) - y^{(i)})$ 是预测误差，$x_j^{(i)}$ 是误差的“权重”——**误差大且特征值大的样本对参数更新的贡献大**。 3. **向量化（Vectorization）**：`X*theta - y` 一次算完所有样本的误差向量；`X^T * 误差` 完成所有参数梯度的同时计算。NumPy 的 BLAS 底层并行远超 Python 显式 for 循环——这是处理大数据集的必要条件。 4. **学习率 α**：步长。太大→震荡甚至发散（J 越来越大）；太小→收敛极慢。课程建议按数量级网格搜索

（0.001, 0.01, 0.1, …）。 5. **收敛条件**：梯度范数接近 0（到达谷底）或达到迭代上限。实践中常监控 $J(\theta)$ 随迭代的变化曲线。

算法伪代码与逻辑解说：随机梯度下降 (Stochastic Gradient Descent, SGD)

伪代码

```
输入: (X, y), alpha, max_iters
输出: theta

1. 初始化 theta = 0
2. 循环 iter = 1..max_iters:
    2.1 随机打乱样本顺序（或随机采样）
    2.2 对每个样本 i（顺序遍历）:
        theta = theta - alpha * (h_theta(x^(i)) - y^(i)) * x^(i)
        // 注意: 每个样本立即更新一次参数，且学习率常随迭代衰减
3. 返回 theta
```

【算法逻辑解说】 1. **与批量的区别**：批量 GD 每步用**全部** m 个样本的梯度；SGD 每步只用一个样本的梯度。SGD 的更新方向是真实梯度的**有噪声估计**。 2. **为什么 SGD 在大数据上更优**：批量 GD 每步代价 $O(mn)$；SGD 每步代价 $O(n)$，且**立即开始改进**。当 m 达百万级，SGD 往往能先于批量 GD 到达可接受解。 3. **噪声是特征而非缺陷**：随机性帮助跳出浅的局部极小（对凸问题无影响，对非凸有益）；代价是收敛路径曲折。常用技巧：学习率按 $1/\text{iter}$ 衰减，保证最终收敛。 4. **Mini-batch SGD（介于两者之间）**：每步用 b 个样本（如 32/64）——现代深度学习的事实标准，兼顾稳定与效率。

关键要点

1. 学习 = 最小化代价函数；MSE 代价源自“高斯噪声 + MLE”的概率假设。
2. 梯度下降（迭代）与正规方程（闭式）是求解线性回归的两种途径，各有适用场景（n 大小、是否需要在线学习）。
3. 特征缩放（归一化到相近范围）能显著加速梯度下降收敛——等高线“圆”时梯度下降走直线。
4. 向量化是工程性能的关键；SGD 适合海量数据。

常见误区与注意事项

- **忘记特征缩放**：特征量纲差异大（面积 0–2000 vs 卧室数 1–5）时，代价等高线是细长椭圆，梯度下降呈锯齿状缓慢收敛。用均值归一化 $x \leftarrow \frac{x-\mu}{\sigma}$。
- **学习率选择不当**：过小收敛慢、过大发散。若 J 在迭代中不降反升，几乎可以断定是 α 过大（或代码有 bug）。
- **把 MSE 用于分类问题**：分类的 y 是离散类别，MSE 会惩罚“正确但数值不同”的预测；分类应用交叉熵（L3）。
- **正规方程遇不可逆 X^TX**：特征线性相关或 $m < n$ 时不可逆；可用伪逆或正则化（L5 的岭回归即解决此问题）。

- **误用梯度下降于非凸问题**：线性回归的 J 是凸函数（唯一全局最优）；但对神经网络等非凸问题，梯度下降只能保证局部最优。

思考题

1. **问题**：设 $m=1$（单样本 (x,y)），推导 SGD 在 $h_\theta(x)=\theta_0+\theta_1 x$ 下的两个更新方程，并解释几何意义。
   - **答案**：$\theta_0 := \theta_0 - \alpha(h_\theta(x)-y)$，$\theta_1 := \theta_1 - \alpha(h_\theta(x)-y)x$。几何意义：误差 $(h_\theta(x)-y)$ 乘以该参数对应的输入分量，即“沿误差下降方向按输入大小成比例地调整权重”。
2. **问题**：正规方程 $\theta=(X^TX)^{-1}X^Ty$ 何时比梯度下降更差?
   - **答案**：当特征数 n 很大（如 $>10^4$）时，X^TX 求逆复杂度 $O(n^3)$ 不可接受；且若需在线（数据不断到达）更新模型，迭代法天然适配，正规方程需要整体重算。
3. **问题**：为什么概率解释中假设 $\epsilon\sim\mathcal{N}(0,\sigma^2)$ 会导出平方损失而非绝对损失?
   - **答案**：因为高斯分布的密度 $p(y|x;\theta)\propto\exp(-\frac{(y-\theta^Tx)^2}{2\sigma^2})$ 取负对数后出现平方项。若假设拉普拉斯噪声则得到绝对损失 $|y-\theta^Tx|$——损失函数的选择对应噪声分布的假设，这是“损失函数从哪来”的深层答案。

Lecture 3: Weighted Least Squares. Logistic Regression. Newton's Method

概述

本讲做两件事：其一，介绍**局部加权线性回归 (LWR)**——一种“非参数”地让线性回归更灵活的方法；其二，进入分类问题，引入**逻辑回归 (Logistic Regression)**——用 Sigmoid 函数把线性输出压缩到 $(0,1)$ 作为概率，并用**梯度上升**与**牛顿法 (Newton's Method)** 两种优化器求解。本讲是“线性模型 → 分类”的关键转折。

核心概念与数学直觉

- **局部加权线性回归 (Locally Weighted Linear Regression, LWR)**：
  - *问题定义*：普通线性回归对全局数据拟合一条直线，欠拟合非线性数据。
  - *直观解释*：预测点 x 时，**只在乎它附近的点**——给附近的训练样本更大权重，远处的权重趋近 0。相当于“为每个查询点拟合一条局部直线”。
  - *数学形式*：最小化加权代价

$$J(\theta)=\frac{1}{2}\sum_{i=1}^{m}w^{(i)}(y^{(i)}-\theta^Tx^{(i)})^2,\qquad w^{(i)}=\exp\left(-\frac{(x^{(i)}-x)^2}{2\tau^2}\right)$$

```
    *   $w^{(i)}$: 第 $i$ 个样本的权重，随其与查询点 $x$ 的距离指数衰减。
    *   $\tau$ (bandwidth): 带宽参数，控制“局部”的范围——$\tau$ 小则只看极近邻，$\tau$ 大则接近全局线性
*   *关键性质*: LWR 是**非参数方法**——训练阶段不保存参数，每次预测都要重新拟合（存储全部数据、预测代价高）;
```

- **逻辑回归 (Logistic Regression)**：

- *问题定义*：二分类，$y \in \{0,1\}$。要求输出“属于类别 1 的概率”。
- *直观解释*：线性回归的 $h_\theta(x) = \theta^T x$ 可能输出任意实数（如 3.2 或 −5），不适合当概率。用 Sigmoid 函数把它“压”进 $(0,1)$：$\theta^T x$ 越大，概率越接近 1。
- *数学形式*：

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1+e^{-\theta^T x}}, \qquad P(y=1|x;\theta) = h_\theta(x), \quad P(y=0|x;\theta) = 1 - h_\theta(x)$$

```
    *   $g(z) = 1/(1+e^{-z})$: **Sigmoid/逻辑函数**。$z \to +\infty$ 时 $g \to 1$; $z \to -\in
    *   *直觉*: 决策边界是 $\theta^T x = 0$ (此时概率 0.5)。Sigmoid 的导数有优美性质: $g'(z) = g(z)
*   *为什么不继续用 MSE? *: 把 $h_\theta(x)$ 换成 Sigmoid 后 MSE 不再是凸函数，梯度下降可能陷于局部最
*   *损失函数（交叉熵，由 MLE 导出）*: 假设 $y \sim \text{Bernoulli}(h_\theta(x))$，最大化对数似然等
```

$$\ell(\theta) = \sum_{i=1}^{m} \left[y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \right]$$

```
    *   *直觉*: 当 $y^{(i)}=1$ 时只有第一项起作用—若模型预测 $h \to 1$（正确），该项 $\to 0$（无惩罚）
*   *梯度上升更新*（对 $\ell$ 最大化）:
```

$$\theta_j := \theta_j + \alpha \sum_{i=1}^{m} (y^{(i)} - h_\theta(x^{(i)})) x_j^{(i)}$$

```
    *   惊人的巧合：形式与线性回归的 LMS 更新**完全一样**（只是 $h$ 换成了 Sigmoid）。这不是偶然，而是**
```

- **牛顿法 (Newton's Method)**：
  - *问题定义*：梯度上升（一阶方法）收敛慢；牛顿法利用**二阶信息（曲率）**加速收敛。
  - *直观解释*：梯度下降像“沿坡走固定步长”；牛顿法用二次函数**局部逼近**目标函数，直接跳到这个二次近似的顶点——更聪明、通常更快（二次收敛）。
  - *数学形式*（最大化 $\ell(\theta)$）：

$$\theta := \theta - H^{-1} \nabla_\theta \ell(\theta), \qquad H_{jk} = \frac{\partial^2 \ell}{\partial \theta_j \partial \theta_k}$$

```
    *   $\nabla_\theta \ell$: 梯度（一阶信息，上升方向）。
    *   $H$: **Hessian 矩阵**（二阶信息，曲率），$n \times n$。
    *   $H^{-1} \nabla \ell$: 用曲率校正步长—曲率大的方向步长小（避免越过），曲率小的方向步长大（快速前
*   *代价*: 每步需计算并求逆 $H$，$O(n^3)$。当 $n$ 不大（如 $<10^3$）且需要高精度解时，牛顿法只需很少迭
*   *逻辑回归中的特殊性质*: $\ell$ 是凹函数，牛顿法保证收敛到全局最优。实践中逻辑回归常用 **IRLS (Iterat
```

算法伪代码与逻辑解说：逻辑回归 + 牛顿法

伪代码

```
输入:
    - 训练数据 (X, y), y ∈ {0,1}
    - 收敛阈值 epsilon, 最大迭代 max_iters

输出:
    - 最优参数 theta

1. 初始化 theta = 0
2. 循环 iter = 1..max_iters:
    2.1 预测概率: h = sigmoid(X * theta)            // 向量化, m 维
    2.2 梯度: grad = X^T * (y - h)                    // 一阶信息
    2.3 Hessian: H = X^T * diag(h .* (1 - h)) * X    // 二阶信息, diag 对角矩阵
    2.4 更新: theta = theta - inv(H) * grad           // 牛顿步
    2.5 若 ||grad|| < epsilon: 终止
3. 返回 theta
```

【算法逻辑解说】 1. **Step 2.1**：`sigmoid(X*theta)` 一次算出所有样本的预测概率 $h_i = g(\theta^T x^{(i)})$。这是模型的前向计算。 2. **Step 2.2 梯度**：$\nabla_\theta \ell = \sum_i (y^{(i)} - h_i) x^{(i)}$——**误差向量 $(y-h)$ 与设计矩阵的乘积**。直觉：若样本 i 被低估（$h_i < y_i$），则误差为正，梯度把 θ 往“增加该样本预测”的方向推。注意这里是对数似然的梯度（上升方向），牛顿法里减去 $H^{-1}grad$ 即朝上升方向走。 3. **Step 2.3 Hessian**：$h_i(1-h_i)$ 是 Sigmoid 在 h_i 处的导数（斜率）——它衡量预测的“不确定性”：$h_i \approx 0.5$ 时 $h_i(1-h_i) \approx 0.25$（最不确定，曲率最大）；$h_i \approx 0$ 或 1 时接近 0（已确定，曲率小）。H 把每个样本的曲率按特征加权累积。 4. **Step 2.4 牛顿步**：$H^{-1}grad$ 同时考虑了方向和曲率。相比梯度上升 $\theta + \alpha \cdot grad$，牛顿法**没有学习率超参数**（曲率自动定步长）且收敛极快（通常 <15 次迭代）。 5. **何时用梯度上升 vs 牛顿法**：n 小、精度要求高 → 牛顿法；n 大（>10^3）→ 梯度上升（Hessian 求逆不可行）。

关键要点

1. 逻辑回归解决分类：Sigmoid 把线性得分映射为概率，决策边界仍是线性的（$\theta^T x = 0$）。
2. 交叉熵损失源自 Bernoulli 假设下的 MLE；它对“过度自信的错误”惩罚极重。
3. 逻辑回归的梯度更新与线性回归 LMS 形式相同——背后是 GLM 的统一理论（L4）。
4. 牛顿法是二阶优化：利用曲率信息，无学习率、二次收敛，但每步 $O(n^3)$。
5. 参数方法（逻辑回归）vs 非参数方法（LWR）：前者训练后丢弃数据、预测快；后者每次预测都需全部数据。

常见误区与注意事项

- **把逻辑回归的输出当“置信度”过度解读**：$h_\theta(x)$ 是条件概率 $P(y=1|x)$ 的估计，但对类别不平衡或分布漂移的数据，校准性（calibration）可能很差。
- **决策边界一定是线性的**：逻辑回归本质是线性分类器；非线性需要特征工程（多项式特征、核方法，见 L7）。
- **用梯度下降最大化 ℓ 时误用“减”梯度**：ℓ 是似然，应**加**梯度（上升）；若在最小化负对数似然，则**减**梯度。符号搞反是常见 bug。
- **牛顿法 Hessian 不可逆**：特征线性相关时 H 奇异；加 λI 正则化（即岭式修正）可解。

- **类别不平衡下盲目用准确率**：$y=1$ 只占 1% 时，全预测 0 也有 99% 准确率；应看 Precision/Recall/PR 曲线（TA Lecture: Evaluation Metrics）。

思考题

1. **问题**：为什么 Sigmoid 函数满足 $g'(z)=g(z)(1-g(z))$，这个性质如何简化逻辑回归梯度推导?
   - **答案**：$g'(z)=\frac{e^{-z}}{(1+e^{-z})^2}=g(z)\cdot\frac{e^{-z}}{1+e^{-z}}=g(z)(1-g(z))$。推导梯度时会出现 $\frac{\partial h}{\partial \theta_j}=h(1-h)x_j$，与交叉熵的 $\frac{y}{h}-\frac{1-y}{1-h}$ 相乘后恰好抵消分母，得到干净的 $(y-h)x_j$。
2. **问题**：牛顿法为什么不需要学习率？它在什么条件下会失败?
   - **答案**：牛顿步 $H^{-1}\nabla$ 已按局部曲率缩放了步长（曲率大→步长小），故无 α。失败条件：H 奇异（不可逆）或目标函数非凹/非凸（可能跳到鞍点或极大点而非所需极值）；对凹的 ℓ 则安全。
3. **问题**：LWR 的带宽 τ 太大或太小时会发生什么?
   - **答案**：$\tau\to\infty$ 时所有权重 ≈ 1，退化为普通线性回归（高偏差/欠拟合）；$\tau\to 0$ 时只有查询点自身权重非零，拟合穿过每个点（高方差/过拟合）。τ 是偏差–方差权衡的旋钮（呼应 L5）。

Lecture 4: Dataset Split; Exponential Family. Generalized Linear Models (GLMs)

概述

本讲先补上工程上至关重要的**数据集划分**纪律（train/dev/test 的用途与数据泄漏风险）；随后进入课程第一个理论高峰：**指数族分布**与**广义线性模型 (GLM)**。GLM 证明了线性回归、逻辑回归、Softmax 回归等看似不同的模型，其实是同一个设计框架（指数族 + 链接函数）的特例——这解释了 L3 中“梯度形式相同”的巧合。

核心概念与数学直觉

- **数据集划分 (Dataset Split)**:
  - *直观解释*：train 集是“课本”（用来学习）；dev/validation 集是“模拟考”（用来调参、选模型）；test 集是“高考”（只考一次，评估最终泛化性能）。
  - *常见比例*：小数据 60/20/20；大数据（百万级）可 98/1/1，因为 dev/test 只需足够统计显著即可。
  - *关键纪律*：dev 与 test 必须反映**真实部署分布**；不得用 test 调参；随机划分前需注意类别分层（stratified split），时序数据用**时间切分**而非随机切分（防泄漏）。
- **指数族分布 (Exponential Family)**:
  - *定义*：一族概率分布，可写成统一形式

$$p(y;\eta)=b(y)\exp\left(\eta^T T(y)-a(\eta)\right)$$

```
    *   $\eta$: **自然参数 (natural parameter)**，控制分布形状。
    *   $T(y)$: **充分统计量 (sufficient statistic)**——通常 $T(y) = y$。
    *   $b(y)$: 基测度 (normalization 的剩余部分)。
    *   $a(\eta)$: **log-partition 函数**，保证分布归一化，即 $a(\eta) = \log \int b(y) e^{\eta
*   *直觉*: 指数族是“一大族常见分布的共同模板”。**伯努利分布**（分类的基础）、**高斯分布**（回归的基础）、多
*   *例子*: 伯努利 $y \sim \text{Bernoulli}(\phi)$ 可写为 $\eta = \log(\phi/(1-\phi))$ (**logit
*   *有用性质*: $E[T(y); \eta] = \frac{\partial a(\eta)}{\partial \eta}$——均值可由 $a$ 的导数直
```

- **广义线性模型 (GLM) 的三个设计假设**：
  1. $y|x;\theta \sim \mathbf{ExponentialFamily}(\eta)$，其中 $\eta = \theta^T x$（**线性假设**：自然参数是特征的线性组合）；
  2. 预测目标是 $h(x) = E[T(y)|x]$（预测充分统计量的期望）；
  3. 自然参数 $\eta = \theta^T x$。
  4. *由假设 2 自动导出链接函数*：$h_\theta(x) = E[y|x] = a'(\eta) = a'(\theta^T x)$——**响应函数 (response function) 是 a'**，其逆为**规范链接 (canonical link)**。
- **GLM 的三大特例（“同一个框架，三个模型”）**：
  1. **线性回归**：$y \sim \mathcal{N}(\mu, \sigma^2)$（高斯），$\eta = \mu$，$a'(\eta) = \eta \Rightarrow h_\theta(x) = \theta^T x$。（恒等链接）
  2. **逻辑回归**：$y \sim \mathbf{Bernoulli}(\phi)$，$\eta = \log \frac{\phi}{1-\phi} \Rightarrow h_\theta(x) = \phi = \frac{1}{1+e^{-\theta^T x}}$（Sigmoid 是 a' 的逆）。（logit 链接）
  3. **Softmax 回归（多分类）**：$y \in \{1, \dots, k\}$，$y \sim \mathbf{Multinomial}(\phi_1, \dots, \phi_k)$ ⇒ 定义 $k-1$ 个自然参数 $\eta_i = \log \frac{\phi_i}{\phi_k}$，反解得

$$\phi_i = \frac{e^{\eta_i}}{\sum_{j=1}^k e^{\eta_j}}, \qquad h_\theta(x) = \begin{bmatrix} P(y=1|x;\theta) \\ \vdots \\ P(y=k|x;\theta) \end{bmatrix} = \frac{1}{\sum_{j=1}^k e^{\theta_j^T x}} \begin{bmatrix} e^{\theta_1^T x} \\ \vdots \\ e^{\theta_k^T x} \end{bmatrix}$$

```
    *   Softmax 是 Sigmoid 的多类推广；分母是归一化因子（softmax 之和恒为 1）。
    *   *直觉*: 把 $k$ 个线性得分 $e^{\theta_j^T x}$ 变成“概率分布”——得分越高概率越大，但用指数放大差距
*   *统一意义*: GLM 告诉我们——**选择“$y$ 服从什么分布” = 选择“用什么损失函数”**。这就是 L2/L3 中梯度形式
```

算法伪代码与逻辑解说：Softmax 回归（多分类 GLM）

伪代码

```
输入：
    - 训练数据 (X, y)，y ∈ {1, ..., k}，k 个类别
    - 学习率 alpha，迭代次数 max_iters

输出：
    - 参数矩阵 Theta ∈ R^(k × (n+1))，每行对应一个类别的权重

1. 初始化 Theta = 0（或小随机值）
2. 循环 iter = 1..max_iters:
    2.1 线性得分：scores = X * Theta^T            // m×k 矩阵
    2.2 Softmax：probs = softmax(scores)           // 每行归一化到概率分布
    2.3 构造指示矩阵：Y_onehot[i, y^(i)] = 1      // m×k 的 one-hot 标签
    2.4 梯度：grad = (1/m) * X^T * (probs - Y_onehot)   // k×(n+1)
    2.5 更新：Theta = Theta - alpha * grad^T
3. 返回 Theta
```

【算法逻辑解说】 1. **Step 2.2 Softmax 归一化**：对第 i 行的得分 $s_j = \theta_j^T x^{(i)}$ 计算 $p_j = e^{s_j} / \sum_l e^{s_l}$——把任意实数得分变成合法的概率分布。数值稳定性技巧：先减去行最大值再取指数（防止 e^{s_j} 溢出）。 2. **Step 2.3 one-hot 编码**：$Y_{\text{onehot}}[i, y^{(i)}] = 1$ 表示“样本 i 的真实类别”。 3. **Step 2.4 梯度**：$(probs - Y_{\text{onehot}})$ 是“预测概率 − 真实 one-hot”的误差矩阵——与 L3 逻辑回归的 $(y - h)$ 完全同构！对每个类别 j、每个特征 l：$\frac{\partial \ell}{\partial \theta_{jl}} = \sum_i (p_j^{(i)} - \mathbf{1}\{y^{(i)} = j\}) x_l^{(i)}$。 4. **Step 2.5 更新**：沿着负梯度下降（最小化交叉熵损失）。Softmax 回归 = 多分类的逻辑回归；决策边界是类别间的**线性分界面**。

关键要点

1. 数据集划分（train/dev/test）是防止评估失真的基本纪律；数据泄漏是隐蔽而致命的错误。
2. 指数族统一了伯努利、高斯、多项等常见分布；$E[T(y)] = a'(\eta)$ 是 GLM 的“发动机”。
3. GLM 三假设（指数族 + 线性自然参数 + 预测充分统计量期望）⇒ 线性回归（恒等链接）、逻辑回归（logit 链接）、Softmax（多项分布）全是特例。
4. “选分布 = 选损失”：高斯→MSE，伯努利→交叉熵。理解这一点比背公式更重要。
5. GLM 家族梯度更新同构：$\theta := \theta + \alpha \sum_i (y^{(i)} - h_\theta(x^{(i)})) x^{(i)}$。

常见误区与注意事项

- **用随机划分处理时序数据**：时间序列/用户行为数据必须按时间切分，否则未来信息泄漏进训练集，评估虚高。
- **Softmax 参数冗余**：k 类只需 $k-1$ 组参数（最后一类可由归一化推出）；但实践中保留 k 组（加正则化）也常用且无碍。
- **误以为 GLM 覆盖所有模型**：GLM 要求 y 属指数族且 η 线性——非线性模型（神经网络、SVM 核方法）在框架之外。
- **忽略数值稳定性**：softmax/交叉熵实现应使用 log-sum-exp 技巧；直接用 e^{s_j} 会溢出。
- **把 dev 与 test 混用**：在 dev 上调参后，dev 性能已“乐观”；test 是唯一的最终裁判。

思考题

1. **问题**：证明伯努利分布属于指数族，并给出 η 与 ϕ 的关系。
   - **答案**：$p(y;\phi)=\phi^y(1-\phi)^{1-y}=\exp(y\log\frac{\phi}{1-\phi}+\log(1-\phi))$。取 $T(y)=y$，$\eta=\log\frac{\phi}{1-\phi}$（logit），$b(y)=1$，$a(\eta)=-\log(1-\phi)=\log(1+e^{\eta})$。反解：$\phi=\frac{1}{1+e^{-\eta}}$——Sigmoid 就是从伯努利指数族形式中自然涌现的。
2. **问题**：为什么说“GLM 的预测函数 $h_\theta(x)=E[y|x]$”决定了链接函数?
   - **答案**：由指数族性质 $E[T(y)]=a'(\eta)$ 和假设 $\eta=\theta^Tx$，得 $h_\theta(x)=a'(\theta^Tx)$。对高斯 $a'(\eta)=\eta$（恒等）；对伯努利 $a'(\eta)=\frac{1}{1+e^{-\eta}}$（Sigmoid）。“响应函数是 a'、链接函数是 $(a')^{-1}$”完全由分布决定，无需手工设计。
3. **问题**：若 y 是“某网页被点击次数”（计数，非负整数），应选用 GLM 的哪个分布与损失？为什么不用高斯/MSE?
   - **答案**：用泊松分布（Poisson，属指数族）→ 泊松回归，损失为泊松负对数似然。因为计数是非负整数且方差随均值变化，高斯假设（对称、常数方差）不合理；MSE 会把预测推向负数。这正是“分布决定损失”的实战例。

Lecture 5: Bias–Variance Tradeoff. Regularization

概述

本讲回答机器学习最核心的问题之一：**为什么模型会出错?** 答案是两种根本不同的错误来源——**偏差 (bias)** 与**方差 (variance)**，它们之间存在此消彼长的权衡。随后引入**正则化 (Regularization)** 作为控制模型复杂度、缓解高方差的系统方法（岭回归、Lasso），并给出**模型选择**的实用工具（交叉验证）。

核心概念与数学直觉

- **偏差–方差分解 (Bias–Variance Decomposition)**：设真实关系 $y=f(x)+\epsilon$（ϵ 为零均值噪声，方差 σ^2），用不同训练集拟合出模型 $\hat{f}$。期望泛化误差可分解为三部分：

$$\mathbb{E}\left[(y-\hat{f}(x))^2\right]=\underbrace{\sigma^2}_{\text{不可约噪声}}+\underbrace{\left(\mathbb{E}[\hat{f}(x)]-f(x)\right)^2}_{\text{Bias}^2}+\underbrace{\mathbb{E}\left[(\hat{f}(x)-\mathbb{E}[\hat{f}(x)])^2\right]}_{\text{Variance}}$$

```
*   $\sigma^2$：**噪声**—数据本身的随机性，任何模型都无法消除（误差下界）。
*   **Bias²**：模型系统性偏离真实函数的程度—**欠拟合**的来源。简单模型（如直线拟合曲线数据）偏差大。
*   **Variance**：模型对不同训练集的敏感程度—**过拟合**的来源。复杂模型（如高次多项式）会“记住”训练集噪声
*   *直观解释*：把模型估计想象成射箭。**偏差**=箭的平均落点偏离靶心多远（系统误差）；**方差**=箭的落点散布多
*   *权衡*：模型复杂度 ↑ ⇒ 偏差 ↓ 但方差 ↑。**最佳复杂度**在两者交点附近（总误差最小）。这就是"tradeoff"的
```

- **正则化 (Regularization)**：在代价函数中加入对参数大小的惩罚，抑制过拟合。
  - *直觉*：惩罚大参数 ⇒ 模型不敢“用力”拟合每个点 ⇒ 曲线更平滑 ⇒ 方差下降（代价是偏差略升）。
  - *岭回归 (Ridge, L_2)*:

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda\sum_{j=1}^{n}\theta_j^2$$

```
    *   $\lambda \ge 0$：**正则化强度**。$\lambda=0$ 退化为普通最小二乘；$\lambda \to \infty$ 时
    *   $L_2$ 惩罚把参数**收缩（shrink）** 但一般不精确置零；闭式解变为 $\theta = (X^TX + \lambda I
*   *Lasso（$L_1$）*：
```

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda\sum_{j=1}^{n}|\theta_j|$$

```
    *   $L_1$ 惩罚倾向于把不重要的 $\theta_j$ **精确置零** ⇒ 自动**特征选择**（稀疏解）。几何直觉：$L_1
*   *贝叶斯视角*：正则化 = 给参数加先验。$L_2$ 对应高斯先验（$\theta_j \sim \mathcal{N}(0, 1/\lambd
```

- **模型选择 (Model Selection)**：
  - *问题*：如何选多项式阶数、λ、特征子集？——**不能看训练误差**（过拟合）也不能看 test 误差（泄漏）。
  - *交叉验证 (Cross-Validation)*：
    - **Hold-out**：train/dev/test 划分，在 dev 上选模型。
    - **k-折交叉验证 (k-fold CV)**：把训练集分成 k 份，轮流用 $k-1$ 份训练、1 份验证，取 k 次验证误差平均。k 常取 5 或 10。
    - *直觉*：CV 是对“模型在新数据上的表现”的**更可靠估计**，因为它多次换用不同训练/验证组合。
  - *特征选择 (Feature Selection)*：前向搜索（逐步加特征）、后向搜索（逐步删特征）；包装法 (wrapper) 用 CV 误差作为选择标准；L_1 正则化可视为嵌入式的自动特征选择。

算法伪代码与逻辑解说：k 折交叉验证 + 岭回归

伪代码

```
输入:
    - 训练数据 (X, y)
    - 候选超参数列表 lambda_list
    - 折数 k

输出:
    - 最优 lambda* 及其 CV 误差估计

1. 把 (X, y) 随机划分为 k 个大小相近的子集 D_1, ..., D_k
2. 对每个 lambda in lambda_list:
    2.1 初始化 cv_error_sum = 0
    2.2 对 i = 1..k:
        2.2.1 训练集 = 除 D_i 外的所有子集; 验证集 = D_i
        2.2.2 theta = 岭回归闭式解 (X_train^T X_train + lambda*I)^{-1} X_train^T y_train
        2.2.3 在 D_i 上计算 MSE, 累加到 cv_error_sum
    2.3 记录 cv_error(lambda) = cv_error_sum / k
3. 返回使 cv_error 最小的 lambda*
```

【算法逻辑解说】 1. **Step 1 划分**：k 折划分保证每个样本恰好被验证一次——CV 误差是“未见数据误差”的无偏近似。 2. **Step 2.2.1**：每折轮流留出一份作验证，避免“用自己的数据考自己”。 3. **Step 2.2.2**：岭回归闭式解中 λI 保证 $X^TX + \lambda I$ **恒可逆**（正定）——正则化的数学红利。 4. **Step 3 选择**：选 CV 误差最小的 λ。注意：CV 误差是对泛化误差的估计，仍可能略微乐观（因为 λ 的选择也“看过”验证数据），但远好于用训练误差。 5. **偏差–方差诊断联动（预告 L13）**：若训练误差高且 CV 误差高 → 高偏差（加容量/特征）；若训练误差低但 CV 误差高 → 高方差（正则化/更多数据）。

关键要点

1. 泛化误差 = 噪声 + Bias^2 + Variance；模型复杂度是它们的旋钮。
2. 偏差–方差权衡是贯穿全课程的主线：LWR 带宽、多项式阶数、λ、网络宽度都是同一个权衡的不同体现。
3. L_2（岭）收缩但不置零；L_1（Lasso）产生稀疏解、做特征选择。
4. 正则化有贝叶斯解释（高斯/拉普拉斯先验 → MAP 估计）。
5. 模型选择必须用交叉验证，绝不看训练误差或 test 误差。

常见误区与注意事项

- **用训练误差选模型**：多项式阶数越高训练误差越低（可到 0），但泛化崩溃。模型选择只认验证误差。
- **正则化时忘记缩放特征**：$\lambda \sum \theta_j^2$ 对不同量纲特征惩罚不均——必须先行标准化特征。
- **对 θ_0（截距）也做惩罚**：通常只惩罚 $\theta_1 .. \theta_n$，不惩罚偏置项（否则模型无法自由平移）。
- **λ 网格过粗**：λ 跨度大（1e−6 到 1e6），应取对数刻度网格。
- **把“方差”与“噪声”混淆**：噪声 σ^2 不可消除，方差是模型对训练集的敏感度——调试时要分清错误来源（L13 的 ML Advice 详述）。

思考题

1. **问题**：一个 100 阶多项式拟合 10 个点，偏差和方差分别如何?
   - **答案**：高方差——模型能精确穿过每个训练点（训练误差≈0），但换训练集拟合结果剧烈变化，验证误差极高；偏差低（模型表达能力足以表示真实函数）。解法：降低阶数、正则化或增加数据。
2. **问题**：L_1 与 L_2 正则化在几何上为何行为不同?
   - **答案**：约束域不同。L_2 约束 $\sum \theta_j^2 \leq t$ 是**球**，最优解可在任意方向非零（收缩）；L_1 约束 $\sum |\theta_j| \leq t$ 是**菱形**（高维是 cross-polytope），其**角点位于坐标轴上**，最优解常落在角点 ⇒ 对应分量恰好为 0（稀疏）。维度越高，L_1 解落在角点（稀疏）的概率越大。
3. **问题**：为什么交叉验证误差仍可能略微乐观地估计真实泛化误差?
   - **答案**：因为 λ（或模型）的选择过程“看过”了所有验证数据——模型选择本身是一种学习。选择出的模型在 CV 中表现好，部分是偶然（选择偏差）。若候选模型极多、数据极少，这种乐观偏差会放大。

Lecture 6: Gaussian Discriminant Analysis. Naive Bayes, Laplace Smoothing

概述

本讲转向**生成式学习算法 (Generative Learning Algorithms)**。与判别模型（逻辑回归直接建模 $P(y|x)$）不同，生成模型**先建模每个类别的数据分布 $P(x|y)$**，再用贝叶斯规则反推 $P(y|x)$。课程介绍两大生成算法：**高斯判别分析 (GDA)**（连续特征，假设各类别服从高斯分布）与**朴素贝叶斯 (Naive Bayes)**（离散/文本特征，假设特征条件独立），并给出**拉普拉斯平滑**解决零概率问题。

核心概念与数学直觉

- **判别 vs 生成**：
  - *判别模型*：直接学习 $P(y|x)$ 或 $x \to y$ 的决策边界（逻辑回归、SVM、神经网络）。只关心边界，不关心数据如何产生。
  - *生成模型*：学习 $P(x|y)$（各类别的分布）与 $P(y)$（先验），然后 `$P(y|x) = \frac{P(x|y) P(y)}{P(x)} \propto P(x|y) P(y)$`（贝叶斯规则；$P(x)$ 对所有 y 相同，可忽略）
  - *直观类比*：判别模型像“只看轮廓就能区分猫狗”的边界；生成模型像“分别记住猫的长相分布和狗的长相分布，再比较新照片更像哪个”。
  - *何时生成更好*：数据量少时（生成模型用更强的结构假设）、需要 $P(x)$（异常检测/生成样本）时；数据充足时判别模型通常更准（边界不需要精确建模分布）。
- **高斯判别分析 (GDA)**：假设 $y \sim \text{Bernoulli}(\phi)$，$x|y=0 \sim \mathcal{N}(\mu_0, \Sigma)$，$x|y=1 \sim \mathcal{N}(\mu_1, \Sigma)$（**两类共享协方差 Σ**）。
  - *参数*：$\phi, \mu_0, \mu_1, \Sigma$，用 MLE 估计：

$$\phi = \frac{1}{m}\sum_i \mathbf{1}\{y^{(i)}=1\}, \quad \mu_k = \frac{\sum_i \mathbf{1}\{y^{(i)}=k\}x^{(i)}}{\sum_i \mathbf{1}\{y^{(i)}=k\}}, \quad \Sigma = \frac{1}{m}\sum_{i=1}^{m}(x^{(i)}-\mu_{y^{(i)}})(x^{(i)}-\mu_{y^{(i)}})^T$$

```
    *   $\phi$：类别 1 的先验比例；$\mu_k$：类别 $k$ 的样本均值；$\Sigma$：加权平均的类内协方差。
*   *决策边界*：$P(y=1|x) = P(y=0|x)$ ⇒ 边界是**二次曲面**；当两类共享 $\Sigma$ 时退化为**线性边界**
*   *多分类 GDA*：每类一个高斯（$\mu_j$ 各不同，$\Sigma$ 共享），Softmax 型决策。
```

- **朴素贝叶斯 (Naive Bayes)**：
  - *问题定义*：文本分类等离散特征问题。特征 $x_j \in \{0,1\}$（如“词典中第 j 个词是否出现”）。
  - *核心假设（朴素）*：**给定 y，各特征条件独立**：

$$P(x_1, \ldots, x_n|y) = \prod_{j=1}^{n} P(x_j|y)$$

```
    *   *直观解释*：假设“出现‘银行’”与“出现‘贷款’”在已知是垃圾邮件后彼此独立——显然不真（相关词常共现），但
*   *参数与 MLE*：$\phi_{j|y=1} = P(x_j=1|y=1)$ 等：
```

$$\phi_{j|y=1} = \frac{\sum_i \mathbf{1}\{x_j^{(i)} = 1 \wedge y^{(i)} = 1\}}{\sum_i \mathbf{1}\{y^{(i)} = 1\}}, \quad \phi_y = \frac{\sum_i \mathbf{1}\{y^{(i)} = 1\}}{m}$$

```
*   *预测*：$P(y=1|x) \propto P(y=1) \prod_j P(x_j|y=1)$，比较两个类别的得分。
*   *文本分类的变体*：二元特征（词是否出现） vs **多项事件模型（multinomial event model)**（考虑词频，
```

- **拉普拉斯平滑 (Laplace Smoothing)**：
  - *问题*：若某个词 x_j 从未在类别 $y = 1$ 的训练样本中出现，MLE 得 $\phi_{j|y=1} = 0$ ⇒ 预测时 $\prod_j P(x_j|y = 1)$ 整体为 0 ⇒ 模型武断否定该类。这是**零概率问题**。
  - *解法*：给计数加 1：

$$\phi_{j|y=1} = \frac{\sum_i \mathbf{1}\{x_j^{(i)} = 1 \wedge y^{(i)} = 1\} + 1}{\sum_i \mathbf{1}\{y^{(i)} = 1\} + 2}$$

```
    *   *直觉*：相当于假设“每个词在每个类别都至少预先见过一次”（伪计数）。对 $k$ 值特征：分子 $+1$、分母 $
*   *为什么有效*：平滑只是**贝叶斯先验**的体现（均匀 Dirichlet 先验下的 MAP 估计）——再次呼应“先验/正则化
```

算法伪代码与逻辑解说：朴素贝叶斯（二元特征 + 拉普拉斯平滑）

伪代码

```
输入:
    - 训练集: m 个样本，每个样本 x^(i) ∈ {0,1}^n（n 个词特征），标签 y^(i) ∈ {0,1}

输出:
    - 参数: phi_y, phi_j|y=0, phi_j|y=1 (j = 1..n)
    - 分类器: 对新样本 x 输出 argmax_y P(y) * Π_j P(x_j|y)

训练阶段:
1. 统计: c1 = 样本中 y=1 的个数; c0 = m - c1
2. phi_y = (c1 + 1) / (m + 2)                    // 平滑后的先验
3. 对 j = 1..n:
    3.1 n11[j] = 样本中 (x_j=1 且 y=1) 的个数; n10[j] = (x_j=1 且 y=0) 的个数
    3.2 phi_j|y=1 = (n11[j] + 1) / (c1 + 2)      // 拉普拉斯平滑 (+1 分子, +2 分母)
    3.3 phi_j|y=0 = (n10[j] + 1) / (c0 + 2)
4. 保存所有 phi

预测阶段:
5. 对新样本 x:
    score_1 = log(phi_y) + Σ_{j: x_j=1} log(phi_j|y=1) + Σ_{j: x_j=0} log(1 - phi_j|y=1)
    score_0 = log(1 - phi_y) + Σ_{j: x_j=1} log(phi_j|y=0) + Σ_{j: x_j=0} log(1 - phi_j|y=0
6. 返回 argmax(score_1, score_0)
```

【算法逻辑解说】 1. **训练阶段本质是数数**：朴素贝叶斯没有迭代优化——MLE 参数只是条件频率计数 + 平滑。这是它极快、极稳的原因。 2. **拉普拉斯平滑的位置 (Step 3.2)**：分子 $+1$ 保证“未见过的词”概率不为零；分母 $+2$ 保证 $\phi_{j|y} + (1 - \phi_{j|y}) = 1$ 仍成立（二元特征）。 3. **预测阶段用 log**：连乘 $\prod_j$ 在 n 大时下溢为 0；取对数把连乘变连加，数值稳定且单调性不变（$\arg\max$ 不变）。这在所有概率模型中都是标准工程技巧。 4. **为什么条件独立假设是关键**：若不独立，$P(x_1, \ldots, x_n|y)$ 需 2^n 个参数；独立假设下只需 $2n$ 个。参数爆炸与“维度灾难”由此缓解。

关键要点

1. 判别模型建模 $P(y|x)$（边界），生成模型建模 $P(x|y)$ 再经贝叶斯规则反推——数据少、需 $P(x)$ 时生成模型占优。
2. GDA 假设高斯：共享 Σ 得线性边界；高斯假设成立时数据效率高于逻辑回归，不成立时逻辑回归更稳健。
3. 朴素贝叶斯的核心是条件独立假设——用强假设换参数效率，文本分类中效果极佳。
4. 拉普拉斯平滑解决零概率问题，本质是均匀先验下的 MAP 估计。
5. 概率模型实现务必用 log 空间防下溢。

常见误区与注意事项

- **混淆 GDA 与逻辑回归的适用性**：GDA 更强假设、更少数据即可；数据量大且非高斯时逻辑回归更安全。**经验法则**：优先逻辑回归（稳健），数据极少且高斯假设合理时用 GDA。
- **忽视平滑**：不平滑的朴素贝叶斯遇到未登录词会直接输出 0 概率，分类完全失效。

- **误以为条件独立假设“必须为真”**：它几乎从不为真，但模型仍常工作良好（偏差小收益大）；只有当依赖关系对分类至关重要时才需更复杂模型。
- **二元特征用词频模型**：二元伯努利模型忽略词频信息；长文本分类用多项事件模型通常更好。
- **对 GDA 的 Σ 不共享**：两类各用各的 Σ 时决策边界变为二次——更灵活但参数翻倍、更易过拟合。

思考题

1. **问题**：证明当两类共享协方差时，GDA 的决策边界是线性的。
   - **答案**：比较 $\log P(y=1|x)$ 与 $\log P(y=0|x)$，高斯密度中的二次项 $-\frac{1}{2}(x-\mu_k)^T\Sigma^{-1}(x-\mu_k)$ 展开后 $x^T\Sigma^{-1}x$ 项在两类间**抵消**（Σ 相同），仅剩 x 的线性项与常数项 $\Rightarrow \log\frac{P(y=1|x)}{P(y=0|x)} = w^Tx+b$，边界 $\{x : w^Tx+b=0\}$ 是超平面。
2. **问题**：训练集中类别 1 有 0 个样本包含词“比特币”，类别 0 有 5 个。不加平滑时预测含“比特币”的邮件会怎样？加平滑后呢？
   - **答案**：不加平滑：$\phi_{\text{btc}|y=1}=0 \Rightarrow P(x|y=1)$ 连乘为 0 ⇒ 该邮件被武断判为类别 0（即使其他特征强烈支持类别 1）。加平滑（$+1$）：$\phi = 1/(c_1+2)$，只略降该词贡献，分类由所有特征共同决定——更稳健。
3. **问题**：为什么朴素贝叶斯即使在条件独立假设明显不成立时仍表现良好？
   - **答案**：分类只需 $\arg\max_y P(y)\prod_j P(x_j|y)$ 的**排序**正确，而不需概率值精确。独立假设带来的偏差在各类别间往往**系统性相似**（互相抵消），且它大幅降低方差（参数少）。偏差小幅上升换方差大幅下降——再次体现偏差–方差权衡。

Lecture 7: Kernels. Support Vector Machines (SVM)

概述

本讲介绍两件互相成就的事：**核方法 (Kernel Methods)**——一种在不显式构造高维特征的情况下让线性模型“隐式”在高维空间工作的技巧；以及**支持向量机 (SVM)**——基于**最大间隔**思想的强大分类器。SVM 的对偶形式与核技巧结合，诞生了“核 SVM”，能高效处理非线性分类。本讲是课程中数学最密集的部分之一。

核心概念与数学直觉

- **特征映射与核技巧**：线性模型无法处理非线性可分数据。思路：把输入 x 映射到高维特征空间 $\phi(x)$，在其中做线性分类。
  - *问题*：$\phi(x)$ 维度可能爆炸（如二次多项式映射有 $O(n^2)$ 维）。
  - *核技巧 (Kernel Trick)*：许多算法（如 SVM 对偶、岭回归）中 $\phi(x)$ **只以内积 $\langle\phi(x),\phi(z)\rangle$ 的形式出现**。若存在函数 $K(x,z)=\langle\phi(x),\phi(z)\rangle$ 可直接高效计算，就无需显式构造 ϕ！
  - *例子*：$K(x,z)=(x^Tz)^2$ 对应二次多项式特征映射——计算是 $O(n)$，而显式 ϕ 是 $O(n^2)$ 维。核技巧 = **免费的高维空间**。
  - *常用核*：
    - **线性核**：$K(x,z)=x^Tz$。

    - **多项式核**：$K(x,z)=(x^Tz+c)^d$。
    - **高斯核 (RBF)**：$K(x,z)=\exp\left(-\frac{\|x-z\|^2}{2\sigma^2}\right)$——对应**无限维**特征空间，衡量两个点的相似度（距离近相似度高）。
  - *合法核的判据（Mercer 定理）*：K 是合法核 $\iff$ 对任意有限点集，核矩阵 $K_{ij}=K(x^{(i)},x^{(j)})$ 是**半正定**的。直觉：核矩阵是“两两相似度”表，必须像内积矩阵一样良定义。
  - *核的运算*：核的和、积、常数倍仍是核——可组合出复杂核。
- **SVM：最大间隔分类器**：
  - *问题定义*：线性二分类。存在无数条分界线都能正确分开数据——SVM 选**间隔最大**的那条。
  - *直观解释*：把分界线想象成“公路”，两侧留出最宽的“缓冲区”（间隔）。间隔越大，对新数据越鲁棒（离边界越远越安全）。SVM 选择使间隔最大的分界线，只由**离边界最近的点**决定——这些点叫**支持向量 (support vectors)**。
  - *函数间隔 vs 几何间隔*：
    - 函数间隔：$\hat{\gamma}^{(i)}=y^{(i)}(w^Tx^{(i)}+b)$（对正确分类样本为正；对 (w,b) 整体缩放会变大——不规范）。
    - 几何间隔：$\gamma^{(i)}=y^{(i)}\left(\frac{w^T}{\|w\|}x^{(i)}+\frac{b}{\|w\|}\right)$——归一化后的函数间隔，**缩放不变**，几何意义是点到超平面的距离。
  - *优化问题（原始形式）*：最大化最小几何间隔：

$$\max_{\gamma,w,b}\gamma\quad\text{s.t.}\quad y^{(i)}(w^Tx^{(i)}+b)\geq\gamma,\ \|w\|=1$$

规范化后等价于：

$$\min_{w,b}\frac{1}{2}\|w\|^2\quad\text{s.t.}\quad y^{(i)}(w^Tx^{(i)}+b)\geq 1,\ \forall i$$

```
    *   $\frac{1}{2}\|w\|^2$：最小化 $\|w\|$ = 最大化间隔（间隔 $= 2/\|w\|$）。
    *   *直觉*：约束“每个点至少在边界外侧”，目标“边界越宽越好”。
*   *软间隔 (Soft Margin)*：数据线性不可分或有噪声时引入**松弛变量 $\xi_i$** 与惩罚 $C$：
```

$$\min_{w,b,\xi}\frac{1}{2}\|w\|^2+C\sum_{i=1}^{m}\xi_i\quad\text{s.t.}\quad y^{(i)}(w^Tx^{(i)}+b)\geq 1-\xi_i,\ \xi_i\geq 0$$

```
    *   $C$：对“越界点”的容忍度。$C$ 大 ⇒ 严格分类（可能过拟合）；$C$ 小 ⇒ 容忍误分（更平滑）。$\xi_i$ 像
*   *对偶形式与核化*：构造拉格朗日，对偶问题只依赖内积 $x^{(i)T} x^{(j)}$（替换为 $K(x^{(i)}, x^{(j)}
```

$$h(x)=\text{sign}\left(\sum_{i\in SV}\alpha_iy^{(i)}K(x^{(i)},x)+b\right)$$

```
    *    *直觉*：新点分类 = 与所有支持向量的“相似度”加权投票。
*    *SMO 算法*：坐标上升思想的特例，每次只优化两个 $\alpha$（闭式解），循环直到收敛——是实践中训练 SVM 的标
```

算法伪代码与逻辑解说：SMO（简化版）

伪代码

```
输入：
    - 训练数据 (X, y)，y ∈ {-1, +1}
    - 核函数 K(·,·)，惩罚参数 C

输出：
    - 拉格朗日乘子 alpha，偏置 b

1. 初始化 alpha = 0, b = 0
2. 循环直到 alpha 收敛（KKT 条件近似满足）：
    2.1 选取一对"违反 KKT 最严重"的乘子 alpha_i, alpha_j（启发式选择）
    2.2 计算误差：E_i = f(x^(i)) - y^(i)，其中 f(x) = Σ_k alpha_k y_k K(x_k, x) + b
    2.3 解析更新 alpha_j（带上下界 L, H 的裁剪）：
         alpha_j_new = alpha_j + y_j (E_i - E_j) / (K_ii + K_jj - 2K_ij)
         alpha_j_new = clip(alpha_j_new, L, H)     // L, H 由 C 与 alpha_i+alpha_j 决定
    2.4 更新 alpha_i = alpha_i + y_i y_j (alpha_j_old - alpha_j_new)
    2.5 更新 b（由支持向量条件）
3. 返回 alpha, b
```

【算法逻辑解说】 1. **为什么对偶**：原始问题是约束优化（难）；对偶问题约束简单（$0 \le \alpha_i \le C$），且目标函数只含核内积——核技巧在此落地。 2. **Step 2.1 选对 (pair)**：SMO 每次只动两个 α（一个也动不了：$\sum \alpha_i y_i = 0$ 约束）。启发式优先选“误差最大”的点对，加速收敛。 3. **Step 2.3 解析更新**：固定其他 α 后，α_j 的优化有闭式解——分子是误差差，分母是核矩阵二阶差（曲率）；裁剪到 $[L, H]$ 保证约束 $0 \le \alpha \le C$ 与 $\sum \alpha_i y_i = 0$ 同时满足。 4. **收敛判定**：所有点满足 KKT 条件（对 $\alpha_i = 0$：点在边界内；$0 < \alpha_i < C$：点在边界上；$\alpha_i = C$：点越界）。KKT 是“最优性”的精确刻画。 5. **预测**：只需支持向量（$\alpha_i > 0$ 的点），其余点不参与——稀疏性让 SVM 预测高效。

关键要点

1. 核技巧：只需内积可计算 ⇒ 隐式高维（甚至无限维）特征空间，计算代价不变。
2. SVM = 最大间隔分类器；间隔大 ⇒ 泛化好（理论上有界，L13 学习理论部分会提）。
3. 软间隔参数 C 是偏差–方差旋钮：C 大→低偏差高方差。
4. 对偶 + KKT：解由支持向量稀疏表示；SMO 是标准训练算法。
5. 高斯核参数 σ 同样控制复杂度：σ 小→决策边界更复杂（高方差）。

常见误区与注意事项

- **不缩放特征直接上核 SVM**：高斯核依赖欧氏距离，量纲大的特征主导相似度——必须先标准化。

- **σ 与 C 一起盲调**：高斯核 SVM 有两个超参数，应网格搜索（对数刻度）。σ 过小→每个点都是孤岛（过拟合）；过大→核退化为线性（欠拟合）。
- **误以为 SVM 输出是概率**：标准 SVM 输出的是到边界的距离（margin），不是概率；需要校准（Platt scaling）才能当概率用。
- **大数据集用 SVM**：核矩阵 $O(m^2)$ 内存、SMO $O(m^2)$~$O(m^3)$ 时间；$m > 10^5$ 时优先考虑线性模型/神经网络。
- **混淆函数间隔与几何间隔的缩放不变性**：函数间隔随 $\|w\|$ 缩放变化，几何间隔不变——优化必须用几何间隔（或固定 $\|w\| = 1$ 的规范化形式）。

思考题

1. **问题**：为什么最大化间隔等价于最小化 $\frac{1}{2}\|w\|^2$?
   - **答案**：几何间隔 $\gamma = 1/\|w\|$（在约束 $y^{(i)}(w^T x^{(i)} + b) \geq 1$ 规范化后，边界到超平面距离为 $1/\|w\|$，总间隔 $2/\|w\|$）。最大化 $2/\|w\| \iff$ 最小化 $\|w\| \iff$ 最小化 $\frac{1}{2}\|w\|^2$（平方仅便于求导）。
2. **问题**：高斯核 $K(x, z) = \exp(-\|x - z\|^2/2\sigma^2)$ 对应无限维特征映射——直觉上它“记住了什么”?
   - **答案**：它衡量点与点的相似度（距离越近越相似）。在训练点上构造的“相似度山峰”（每个支持向量一个峰），叠加后形成任意复杂的决策曲面。σ 控制峰宽：σ 小→峰窄→只影响极近的点→边界锯齿化（高方差）。
3. **问题**：线性不可分数据上，为何软间隔 SVM 仍可能优于强制硬间隔?
   - **答案**：硬间隔要求所有点严格分开——对噪声点会“委曲求全”形成病态边界（高方差、泛化差）。软间隔允许少数点越界（付出 $C\xi_i$ 代价），换来更平滑、间隔更大的边界——用少量训练误差换更小的泛化误差（偏差–方差权衡）。

Lecture 8: K–Means. Gaussian Mixture Models (GMM). Expectation Maximization (EM)

概述

本讲进入无监督学习：**K–Means**（最经典的硬聚类算法）与**高斯混合模型 GMM + 期望最大化 EM**（软聚类/密度估计）。核心思想：数据存在**隐藏结构（潜在变量）**——每个样本属于哪个簇是我们看不到的。EM 算法为“带隐藏变量的最大似然估计”提供通用框架，是课程中最重要的算法范式之一。

核心概念与数学直觉

- **K–Means 聚类**：
  - *问题定义*：给定无标签数据 $\{x^{(1)}, \ldots, x^{(m)}\}$，把数据分成 K 个簇，使簇内点尽量接近。
  - *直观解释*：把每个簇想象成一个“引力中心”（质心）。算法交替做两件事：把每个点指派给最近的质心；把质心移动到其簇内点的均值。反复迭代直到稳定。
  - *数学形式*：最小化簇内平方距离和（distortion）：

$$J(c,\mu)=\sum_{i=1}^{m}\|x^{(i)}-\mu_{c^{(i)}}\|^2$$

```
    *   $c^{(i)} \in \{1,\dots,K\}$：样本 $i$ 的簇指派（潜在变量）。
    *   $\mu_k$：簇 $k$ 的质心（均值）。
*   *性质*：坐标下降（每次更新 $c$ 或 $\mu$ 都使 $J$ 不增）⇒ **保证收敛到局部最优**（不同初始化可能不同结
```

- **高斯混合模型 (GMM)**：
  - *问题定义*：数据由 K 个高斯分布混合生成，但不知道每个样本来自哪个高斯。
  - *生成过程*：先按 ϕ_j 抽类别 $z^{(i)}\sim\text{Multinomial}(\phi)$，再抽 $x^{(i)}|z^{(i)}=j\sim\mathcal{N}(\mu_j,\Sigma_j)$。$z$ 是**隐藏/潜在变量**。
  - *似然*：

$$\ell(\phi,\mu,\Sigma)=\sum_{i=1}^{m}\log\sum_{j=1}^{K}\phi_j\cdot\frac{1}{(2\pi)^{d/2}|\Sigma_j|^{1/2}}\exp\left(-\frac{1}{2}(x^{(i)}-\mu_j)^T\Sigma_j^{-1}(x^{(i)}-\mu_j)\right)$$

```
    *   注意 $\log$ 在**求和**（混合）外面——无法像单高斯那样闭式求解；直接对 $\ell$ 求导置零得不到闭式解
*   *K-Means vs GMM*：K-Means 是“硬”指派（每个点属于一个簇）；GMM 是“软”指派（每个点以概率属于各簇），且
```

- **EM 算法（期望最大化）**：
  - *核心思想*：直接最大化 ℓ 太困难（log 套在求和里）。EM 用两步迭代绕开：
    1. **E 步**：固定参数，计算每个样本属于各簇的**后验概率**（“期望”）：$w_j^{(i)}=P(z^{(i)}=j|x^{(i)};\phi,\mu,\Sigma)$。
    2. **M 步**：用这些后验概率作为权重，最大化**加权对数似然**，得到新参数（“最大化”）。
  - *理论保障（Jensen 不等式）*：EM 构造似然的下界（ELBO），E 步使下界在该点**紧**（相等），M 步**提升下界** ⇒ 每次迭代 ℓ 单调不减 ⇒ 收敛到（局部）最优。这是“优化难目标”的通用策略：*优化一个容易的下界*。
  - *GMM 的 EM 更新（M 步）*：

$$\phi_j=\frac{1}{m}\sum_i w_j^{(i)},\qquad \mu_j=\frac{\sum_i w_j^{(i)}x^{(i)}}{\sum_i w_j^{(i)}},\qquad \Sigma_j=\frac{\sum_i w_j^{(i)}(x^{(i)}-\mu_j)(x^{(i)}-\mu_j)^T}{\sum_i w_j^{(i)}}$$

```
    *   形式与 GDA/朴素贝叶斯的 MLE 相同，只是每个样本按 $w_j^{(i)}$ **加权**—权重就是“它属于簇 $j$
*   *EM 的更一般表述（L8 后半）*：E 步计算 $Q(\theta, \theta^{old}) = \sum_i \sum_z Q_i(z) \log
```

算法伪代码与逻辑解说：K-Means

伪代码

```
输入:
    - 数据 X (m×d), 簇数 K, 最大迭代 max_iters

输出:
    - 质心 mu_1..mu_K, 簇指派 c^(1)..c^(m)

1. 初始化: 随机选 K 个样本作为初始质心 mu_1..mu_K
2. 循环 iter = 1..max_iters:
    2.1 指派步骤 (Assignment):
        对每个样本 i: c^(i) = argmin_j ||x^(i) - mu_j||^2
    2.2 更新步骤 (Update):
        对每个簇 j: mu_j = (1/|S_j|) * Σ_{i∈S_j} x^(i)   // S_j = 指派到 j 的样本集合
    2.3 若指派不再变化: 终止
3. 返回 (mu, c)
```

【算法逻辑解说】 1. **Step 2.1 指派**：每个点认领“最近的质心” —— 对应 J 对 c 的坐标下降。复杂度 $O(mKd)$。 2. **Step 2.2 更新**：质心移到簇内均值 —— 对应 J 对 μ 的坐标下降。均值是“最小化平方距离和的中心”的闭式解。 3. **单调性**：两步都使 J 不增 ⇒ 收敛；但 J 非凸 ⇒ 收敛到**局部最优**。对策：多次随机初始化 + 选最小 J；或用 K-Means++ 初始化（让初始质心彼此远离）。 4. **如何选 K**：肘部法则（画 J vs K，找拐点）；或根据下游任务/业务约束选择。 5. **与 EM 的关系**：K-Means 是 GMM 的 EM 在“硬指派 + 单位协方差”下的极限特例（$w_j^{(i)}$ 变成 0/1）。

算法伪代码与逻辑解说：GMM 的 EM

伪代码

```
输入:
    - 数据 X (m×d), 簇数 K (高斯数), 最大迭代 max_iters, 收敛阈值 epsilon

输出:
    - 混合权重 phi_1..phi_K, 均值 mu_j, 协方差 Sigma_j

1. 初始化 phi, mu, Sigma (如 K-Means 结果作为 mu 初值)
2. 循环 iter = 1..max_iters:
    // ── E 步: 计算后验 (软指派) ──
    2.1 对每个 i, j:
        w_j^(i) = phi_j * N(x^(i); mu_j, Sigma_j) / Σ_l phi_l * N(x^(i); mu_l, Sigma_l)
    // ── M 步: 加权最大似然 ──
    2.2 phi_j = (1/m) Σ_i w_j^(i)
    2.3 mu_j = Σ_i w_j^(i) x^(i) / Σ_i w_j^(i)
    2.4 Sigma_j = Σ_i w_j^(i) (x^(i)-mu_j)(x^(i)-mu_j)^T / Σ_i w_j^(i)
    2.5 计算对数似然 ell; 若 |ell_new - ell_old| < epsilon: 终止
3. 返回 (phi, mu, Sigma)
```

【算法逻辑解说】 1. **E 步 (Step 2.1)**：$w_j^{(i)} = P(z^{(i)} = j | x^{(i)})$ 由贝叶斯规则得到——分子是“簇 j 的先验 × 该簇高斯密度”，分母是归一化（全概率）。这是**软版本**的 K-Means 指派：不是 argmax 而是概率分布。 2. **M 步 (Step 2.2–2.4)**：形式上与 GDA 的 MLE 相同，只是每个样本按 $w_j^{(i)}$ 加权——权重大（很可能属于簇 j）的样本对 μ_j, Σ_j 影响大。 3. **收敛**：EM 保证对数似然单调不减；实践中监控 ℓ 变化。初始化影响结果

（局部最优），常用 K-Means 结果初始化 μ。 4. **为什么 EM 而非直接梯度**：E/M 两步都有闭式解，无需选择学习率；且天然处理“软指派”的不确定性。对比：直接对 ℓ 做梯度上升也可行但更慢、更繁琐。 5. **潜在变量视角**：$z^{(i)}$ 是“看不见的簇标签”——EM 是“用期望补全隐藏信息再优化”的通用范式，L15 的因子分析、隐马尔可夫模型都复用此框架。

关键要点

1. K-Means：硬聚类、坐标下降、保证收敛到局部最优；用多次初始化缓解。
2. GMM：软聚类 + 密度估计；似然中 log 与求和纠缠导致无闭式解 → 引入 EM。
3. EM = E 步（算后验/期望）+ M 步（最大化加权似然），单调提升似然下界（Jensen），收敛到局部最优。
4. K-Means 是 EM 的硬指派极限特例；两者共享“指派-更新”的交替结构。
5. 潜在变量建模是处理“数据生成过程含隐藏结构”的通用思想（贯穿无监督学习与后续模型）。

常见误区与注意事项

- **K-Means 假设球形簇**：基于欧氏距离的均值，对细长/异形簇效果差（各向异性协方差的 GMM 更合适）。
- **K-Means 前不标准化特征**：量纲差异直接扭曲距离计算。
- **EM 收敛到局部最优就罢手**：应多次随机初始化选最高似然；单次运行结果不可靠。
- **协方差奇异（某簇样本少于维度）**：Σ_j 不可逆导致 $N(x;\mu_j,\Sigma_j)$ 溢出——加小对角项（$\Sigma_j+\epsilon I$）或限制协方差结构（对角/共享）。
- **混淆 E 步与 M 步**：E 步是**概率计算**（固定参数算后验），M 步是**参数更新**（固定后验优化参数）——两者角色不可颠倒。
- **忽视 log 空间**：GMM 的 $w_j^{(i)}$ 计算涉及指数与归一化，务必在 log 空间计算后验再 exp（防下溢）。

思考题

1. **问题**：为什么 GMM 的似然函数无法像单高斯那样闭式求解，而 EM 可以绕过?
   - **答案**：单高斯 $\log\prod_i \mathcal{N}(x^{(i)};\mu,\Sigma)$ 中 log 直接作用在每个高斯密度上，求导闭式可得。GMM 的 $\log\prod_i\sum_j \phi_j\mathcal{N}(\cdot)$ 中 log 在**混合求和之外**，求导后 μ_j 的方程与所有簇的 z 纠缠，无法解耦。EM 用 E 步的期望把“每个样本对每个簇的归属”固定下来，使 M 步退化为可闭式求解的加权 MLE。
2. **问题**：EM 每轮迭代保证 $\ell(\theta^{new})\geq\ell(\theta^{old})$，请用 Jensen 不等式直观解释。
   - **答案**：定义下界 $\mathcal{L}(\theta)=\sum_i\sum_z Q_i(z)\log\frac{p(x^{(i)},z;\theta)}{Q_i(z)}$，因 $\log$ 是凹函数，Jensen 给出 $\ell(\theta)\geq\mathcal{L}(\theta)$。E 步选 $Q_i(z)=P(z|x^{(i)};\theta^{old})$ 使 $\mathcal{L}(\theta^{old})=\ell(\theta^{old})$（下界紧）；M 步最大化 $\mathcal{L}$ 得 $\theta^{new}\Rightarrow\ell(\theta^{new})\geq\mathcal{L}(\theta^{new})\geq\mathcal{L}(\theta^{old})=\ell(\theta^{old})$。
3. **问题**：K-Means 与 GMM 的 EM 有何异同? 什么场景下 GMM 明显优于 K-Means?
   - **答案**：相同：交替“指派（E）+ 更新（M）”，都收敛到局部最优。不同：K-Means 硬指派 0/1、无概率输出、假设簇为“球”；GMM 软指派概率、估计密度 $P(x)$、协方差可各向异性。场景：需要不确

定性估计（软指派）、密度估计/异常检测（$P(x)$ 阈值）、簇形状不规则时，GMM 胜出；大规模高维数据上 K-Means 更快更简单。

Lecture 9: Decision Trees

概述

本讲介绍**决策树 (Decision Trees)**——一种直观、可解释、无需特征缩放的分类/回归模型。核心问题：如何自动选择“先问哪个特征、按什么阈值切分”来构建树？答案是**信息论准则**：每次分裂选择使“混乱度”下降最多的特征（信息增益 / Gini 不纯度）。本讲还讨论树的过拟合控制（预剪枝/后剪枝）与集成（随机森林）。

核心概念与数学直觉

- **决策树结构**：一棵树由内部节点（特征测试，如“年龄 < 30？”）与叶节点（预测标签）组成。预测 = 沿根到叶的路径走一遍。
  - *直观解释*：像“二十问”游戏——每次问一个最能区分答案的问题，逐步缩小范围，直到确定答案。
  - *优点*：可解释（人类可读的规则集）、天然处理混合类型特征、不需特征缩放、对非线性关系友好。
  - *缺点*：单棵树易过拟合、不稳定（数据小扰动→树结构大变）；对特征交互的表达依赖深树。
- **熵与信息增益 (Entropy & Information Gain)**：
  - *熵 (Entropy)*：衡量集合的“混乱度/不确定性”：

$$H(S) = -\sum_{c=1}^{C} p_c \log_2 p_c$$

```
    *   $p_c$: 集合 $S$ 中类别 $c$ 的比例。
    *   *直觉*: 全部同一类 ⇒ $p=1$ ⇒ $H=0$ (最“纯”); 均匀分布 ⇒ $H = \log_2 C$ (最混乱)。熵 = 编码一
*   *信息增益 (Information Gain)*: 按特征 $A$ 分裂后熵的**减少量**:
```

$$\text{Gain}(S, A) = H(S) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

```
    *   $S_v$: 特征 $A$ 取值 $v$ 的样本子集。
    *   *直觉*: 分裂后子集的加权熵越小 (越纯), 信息增益越大—**每次选信息增益最大的特征分裂** (贪心)。
*   *Gini 不纯度* (CART 用): $\text{Gini}(S) = 1 - \sum_c p_c^2$—随机抽取两个样本类别不同的概率。G
*   *回归树*: 分裂目标改为最小化子集内**方差** (MSE 下降量), 叶节点输出均值。
```

- **过拟合控制**：
  - *预剪枝 (Pre-pruning)*：限制最大深度、最小叶节点样本数、最小分裂增益。
  - *后剪枝 (Post-pruning)*：先长满树，再从底向上评估“剪掉子树换成叶”是否提升验证集性能。

- *随机森林 (Random Forest)*：Bagging + 随机特征子集——训练多棵（在自助采样子集上、每层只用随机子集特征）的树，投票集成。随机性降低树间相关性 ⇒ 大幅降方差。

算法伪代码与逻辑解说：ID3/CART 式决策树构建

伪代码

```
输入：
    – 训练集 S（样本+标签），特征集 F，超参数（max_depth, min_samples_split, min_gain）

输出：
    – 决策树 T

函数 BuildTree(S, F, depth)：
1. 若满足停止条件（depth >= max_depth 或 |S| < min_samples_split 或
    S 中所有样本同类别 或 F 为空）：
    1.1 返回叶节点，标签 = S 中多数类（回归为均值）
2. 对每个特征 f ∈ F（及每个候选分裂点/阈值）：
    2.1 计算分裂后的信息增益 Gain(S, f)（或 Gini 下降 / MSE 下降）
3. 选择增益最大的特征 f*（及最优阈值）
4. 若 Gain(S, f*) < min_gain：返回叶节点（多数类）
5. 用 f* 把 S 划分为子集 S_1, ..., S_v
6. 对每个子集 S_v：child_v = BuildTree(S_v, F \ {f*}, depth+1)
7. 返回内部节点（f*, children）

预测：沿根到叶的路径，返回叶节点标签
```

【算法逻辑解说】 1. **Step 1 停止条件**：防止无限生长与过拟合。叶节点输出**多数类**（分类）或**均值**（回归）——叶是“局部常数模型”。 2. **Step 2–3 贪心分裂**：在每个节点独立地选“当前最有区分力的特征”——这是**贪心**策略（不回溯、不考虑未来分裂），计算高效但可能错过全局最优树（NP-hard 问题的实用近似）。 3. **连续特征**：按值排序后尝试相邻点中点为阈值，选增益最大的阈值。 4. **Step 6 递归**：分治——每个子问题与父问题同构，天然递归实现。特征不重复使用（ID3 风格）或可重复使用（CART 风格，对连续特征常重复）。 5. **预测复杂度**：$O(\text{深度})$——极快，适合低延迟推理。 6. **与偏差–方差的关系**：单棵树高方差（换数据大变）；剪枝/限制深度=正则化（升偏差降方差）；随机森林=集成降方差。

关键要点

1. 决策树 = 递归划分特征空间；分裂准则（熵/Gini/MSE）衡量“纯化程度”。
2. 熵是信息论的“混乱度”度量：信息增益 = 分裂带来的不确定性减少。
3. 树的主要敌人是过拟合：深度、叶大小、min_gain 是正则化旋钮；后剪枝用验证集。
4. 随机森林通过 Bagging + 随机特征子集大幅降低方差，是树的实用形态。
5. 树的可解释性（规则集）是相对神经网络的核心优势。

常见误区与注意事项

- **不设停止条件导致过拟合**：满树在训练集误差 0，但泛化差。务必限制深度/叶大小或剪枝。

- **用信息增益做多值特征时偏好取值多的特征**：ID3 的 Gain 偏向取值数多的特征（如 ID 列）；可用增益率 (Gain Ratio) 或 CART 的 Gini 缓解。
- **忽视类别不平衡**：多数类主导叶标签；可用加权分裂或过采样。
- **在需要概率输出的场景直接用树**：单树输出是硬标签；概率需要叶内比例（校准差）或改用梯度提升树（GBDT 可输出分数）。
- **树的“不稳定”不等于“不好”**：单树方差大是特性；集成（RF/GBDT）才是树的完整形态，实践中几乎总是用集成。

思考题

1. **问题**：为什么熵的公式是 $-\sum p_c \log p_c$？它如何度量“编码成本”？
   - **答案**：信息论中，编码概率 p 的事件最优需 $\log_2(1/p) = -\log_2 p$ 比特（香农）。熵 = 各事件编码长度的期望。类别越不确定（分布越均匀），平均编码越长——熵高 = 混乱。
2. **问题**：一个数据集的标签全为“猫”，另一个猫狗各半，哪个熵大？分裂后信息增益可能为负吗？
   - **答案**：前者 $H = -1 \log 1 = 0$，后者 $H = -2 \times 0.5 \log 0.5 = 1$。信息增益理论上 ≥ 0（H 是凹函数，加权平均子集熵 $\leq$ 原熵，Jensen 不等式）；但若用验证集评估或分裂准则不一致，观测值可能“虚增”不提升真实泛化——这也是需要剪枝校验的原因。
3. **问题**：为什么随机森林要随机选特征子集，而不只是 Bagging？
   - **答案**：若所有树都用全部特征，Bagging 后树间仍高度相关（强特征被所有树首选），集成方差下降有限。随机特征子集迫使树“各看各的角度”，降低相关性——集成效果随树间不相关性提升。这是“多样性是集成之本”的体现。

Lecture 10: Boosting（AdaBoost）

概述

本讲介绍与 Bagging 思路相反的集成方法——**Boosting**。Bagging 并行训练独立模型取平均（降方差）；Boosting **串行**训练一系列“弱学习器”，每个新学习器**聚焦上一个犯错的样本**，最终加权组合（降偏差）。代表算法 **AdaBoost**：用带权重的样本训练弱分类器，按错误率分配话语权，逐步把弱学习器提升为强学习器。

核心概念与数学直觉

- **集成学习的两种哲学**：
  - *Bagging*（并行，如随机森林）：独立训练、投票平均 → 降**方差**。适合高方差模型（深树）。
  - *Boosting*（串行，如 AdaBoost/GBDT）：逐步修正错误 → 降**偏差**。适合高偏差模型（浅树/弱分类器）。
  - *直觉*：Bagging 像“多个独立专家投票”（每人独立判断，抵消随机错误）；Boosting 像“师徒相传”——每个新徒弟专攻师父的短板。
- **AdaBoost 的核心机制**：

- *样本权重*：每个训练样本有权重 $D^{(i)}$，初始均匀 $1/m$。每轮训练后，**被分错的样本权重增大、分对的减小**——下一轮的弱分类器被迫“重视”难样本。
- *分类器权重*：每轮弱分类器 h_t 的话语权 α_t 由其加权错误率 ϵ_t 决定：

$$\epsilon_t = \sum_{i:h_t(x^{(i)})\neq y^{(i)}} D_t^{(i)}, \qquad \alpha_t = \frac{1}{2}\ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$$

```
    *   $\epsilon_t$：第 $t$ 轮弱分类器的加权错误率。
    *   $\alpha_t$：**话语权**——错误率越低（$\epsilon_t \to 0$）话语权越大；随机猜（$\epsilon_t =
*   *权重更新*：
```

$$D_{t+1}^{(i)} = \frac{D_t^{(i)}\exp(-\alpha_t y^{(i)} h_t(x^{(i)}))}{Z_t}$$

```
    *   $y^{(i)} h_t(x^{(i)})$：正确分类时 $= +1$（权重乘 $e^{-\alpha_t} < 1$，减小）；错误时 $= -
    *   $Z_t$：归一化因子，保证 $\sum_i D^{(i)}_{t+1} = 1$。
*   *最终预测*：加权投票：
```

$$H(x) = \text{sign}\left(\sum_{t=1}^{T}\alpha_t h_t(x)\right)$$

```
*   *理论保证*：若每轮 $\epsilon_t < 0.5$（略好于随机），AdaBoost 的训练误差**指数级下降**：
    $\hat{\epsilon} \le \exp\left(-2 \sum_{t=1}^{T} \left(\frac{1}{2} - \epsilon_t\right)^2
*   *为什么 Boosting 能降偏差*：弱学习器（如深度 1 的决策树桩）单个偏差大；串行叠加使最终模型能表达复杂边界
*   *与梯度下降的联系*：AdaBoost 可看作在**指数损失** $L(y, f(x)) = e^{-y f(x)}$ 上的**前向分步加法建
```

算法伪代码与逻辑解说：AdaBoost

伪代码

```
输入:
    - 训练集 (X, y), y ∈ {-1, +1}
    - 弱学习器算法 WeakLearner (如深度 1 决策树桩)
    - 轮数 T

输出:
    - 强分类器 H(x) = sign( Σ_t α_t h_t(x) )

1. 初始化样本权重 D^(i) = 1/m (i = 1..m)
2. 对 t = 1..T:
    2.1 用权重 D 训练弱学习器 h_t (最小化加权错误率)
    2.2 计算加权错误率: ε_t = Σ_{i: h_t(x^(i)) ≠ y^(i)} D^(i)
    2.3 若 ε_t >= 0.5: 令 h_t 反转 (或终止/重启权重)
    2.4 计算话语权: α_t = 0.5 * ln((1 - ε_t) / ε_t)
    2.5 更新权重: D^(i) = D^(i) * exp(-α_t * y^(i) * h_t(x^(i))), 再除以 Z_t 归一化
3. 返回 H(x) = sign( Σ_t α_t h_t(x) )
```

【算法逻辑解说】 1. **Step 1**：所有样本一视同仁（均匀权重）。 2. **Step 2.1**：弱学习器必须能处理**样本权重**——决策树桩按权重计算 Gini/误差，即“重样本犯错代价更高”。 3. **Step 2.3 关键保障**：若 $\epsilon_t \geq 0.5$（不优于随机），翻转 h_t（预测取反）使其错误率 ≤ 0.5；否则 $\alpha_t \leq 0$ 无意义。 4. **Step 2.4**：α_t 是“信任度”。注意 $\epsilon_t \to 0$ 时 $\alpha_t \to +\infty$——完美分类器话语权极大（但此时权重更新 Z_t 会异常，实践中加小 ϵ 或直接用强学习器）。 5. **Step 2.5**：错误样本权重乘 e^{α_t}、正确样本乘 $e^{-\alpha_t}$——下一轮弱学习器被迫聚焦难样本。$Z_t = \sum_i D^{(i)} \exp(-\alpha_t y^{(i)} h_t(x^{(i)}))$ 保证概率归一。 6. **Step 3 投票**：强分类器 = 带权投票。只取 $\mathbf{sign}$ 得硬标签；去掉 sign 的实数值 $f(x) = \sum_t \alpha_t h_t(x)$ 可当置信分数用（校准需额外处理）。

关键要点

1. Boosting 串行纠错降偏差；Bagging 并行平均降方差——适用场景不同。
2. AdaBoost 三要素：样本权重（聚焦难样本）、话语权 α_t（信任度）、加权投票（组合）。
3. 弱学习器“略好于随机”即可，理论保证训练误差指数下降。
4. AdaBoost ≈ 指数损失上的前向分步加法建模——与梯度下降同源的优化视角。
5. 现代实战中 **GBDT/XGBoost/LightGBM** 是 Boosting 的主流形态（回归树 + 二阶梯度近似 + 正则化）。

常见误区与注意事项

- **对噪声数据用 AdaBoost**：AdaBoost 会把权重集中到异常点，最终被噪声带偏（过拟合噪声）。实践：限制 T、用早停（监控验证误差）。
- **弱学习器太强**：若 h_t 每轮都近乎完美，$\epsilon_t \approx 0$，α_t 爆炸且后续轮次权重失衡——Boosting 的意义在于“弱”学习器的叠加。
- **忽略样本权重**：决策树实现必须支持加权分裂；朴素实现（忽略权重）的 AdaBoost 完全失效。
- **把 α_t 直接当概率**：α_t 是话语权不是概率；H 的实数值输出需 Platt 校准才能解释为概率。
- **混淆 AdaBoost 与 Bagging 的适用模型**：AdaBoost 配弱模型（树桩）；随机森林配强模型（深树）——用反了效果差。

思考题

1. **问题**：若第 t 轮弱分类器错误率 $\epsilon_t = 0.4$，计算 α_t，并说明其含义。
   - **答案**：$\alpha_t = \frac{1}{2}\ln\frac{0.6}{0.4} \approx 0.203$——比随机（0.5）好，话语权为正；下一轮错误样本权重乘 $e^{0.203} \approx 1.225$，正确样本乘 $e^{-0.203} \approx 0.816$——难样本被放大约 1.5 倍。
2. **问题**：为什么 AdaBoost 对噪声敏感？如何缓解？
   - **答案**：错误样本权重每轮指数放大，噪声点（本就不可能被正确分类）权重会主导后续训练，弱学习器被迫拟合噪声 ⇒ 过拟合。缓解：限制轮数 T（早停）、在验证集上监控、使用带“噪声鲁棒”损失的变体（如修改损失为截断形式）。
3. **问题**：AdaBoost 与梯度下降有什么深层联系？
   - **答案**：把强分类器看成函数 $f(x) = \sum_t \alpha_t h_t(x)$ 的逐步构造：每轮选择 (α_t, h_t) 使指数损失 $L = \sum_i e^{-y^{(i)} f(x^{(i)})}$ 下降最快——这正是**函数空间中的坐标下降/前向分步加法建模**。GBDT 把“指数损失”推广到任意可微损失，用负梯度作为拟合目标，即“任意损失的 Boosting”。

Lecture 11: Neural Networks 1（前向传播与基础架构）

概述

本讲把“神经元”概念形式化：神经网络是**多层非线性函数的复合**，每一层是“线性变换 + 非线性激活”。讲清楚前向传播（forward propagation）如何把输入变换为输出、为什么需要非线性激活、如何向量化实现，以及逻辑回归如何作为“单神经元”特例嵌入框架。

核心概念与数学直觉

- **从逻辑回归到神经网络**：逻辑回归 $h = g(\theta^T x)$ 是一个“单神经元”：输入 x → 线性加权 $\theta^T x$ → 非线性压缩 g。神经网络 = **多个这样的神经元分层堆叠**：前一层的输出作为后一层的输入。
  - *直观解释*：第 1 层神经元学习“低级特征”（如像素边缘），第 2 层组合成“中级特征”（如形状），更高层组合成“高级概念”（如人脸）——**特征的层级化自动学习**，无需手工特征工程。
  - *数学形式（前向传播）*：设 $a^{[0]} = x$，对层 $l = 1..L$：

$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}, \qquad a^{[l]} = g^{[l]}(z^{[l]})$$

```
    *   $W^{[l]}$：第 $l$ 层权重矩阵（行=该层神经元数，列=上一层神经元数）。
    *   $b^{[l]}$：偏置向量；$z^{[l]}$：线性组合（pre-activation）。
    *   $g^{[l]}$：**激活函数**（非线性）；$a^{[l]}$：该层输出（activation）。
*   *为什么必须非线性*：若所有 $g$ 都是恒等（线性），多层复合仍是线性函数 $W_{total} x$——**深层毫无意义*
```

- **常见激活函数**：

激活	公式	输出范围	特点
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	$(0,1)$	历史经典；饱和区梯度消失；输出非零中心
tanh	$\tanh(z)$	$(-1,1)$	零中心；仍会饱和
ReLU	$\max(0,z)$	$[0,\infty)$	计算快、缓解梯度消失；负数侧梯度为 0（死亡神经元）
Leaky ReLU	$\max(0.01z, z)$	$\mathbb{R}$	缓解死亡神经元
Softmax（输出层）	$\frac{e^{z_j}}{\sum_k e^{z_k}}$	概率分布	多分类输出层

- **输出层与损失的选择（GLM 思想的延续）**：
  - 回归 → 线性输出 + MSE（或 Huber）。
  - 二分类 → Sigmoid 输出 + 交叉熵。
  - 多分类 → Softmax 输出 + 交叉熵。
  - *直觉*：输出层激活函数 + 损失函数 = 对 y 分布假设的体现（呼应 L4 的 GLM 框架）。
- **向量化与批处理**：把 m 个样本堆成矩阵 X（$n \times m$），一层计算 $Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$——利用 BLAS 并行。批处理 (mini-batch) 是内存与梯度噪声的平衡点。
- **网络容量与过拟合**：宽度（每层神经元数）、深度、激活类型都是容量旋钮——容量大则易过拟合，配 Dropout/权重衰减等正则化（L12/L13 展开）。

算法伪代码与逻辑解说：前向传播（向量化）

伪代码

```
输入:
    - 输入 X（n×m，n 特征，m 样本）
    - 网络参数 {W^[l], b^[l]}_{l=1..L}，激活函数 g^[l]

输出:
    - 每层激活 A^[l]，最终预测 A^[L]（即 h）

1. A^[0] = X
2. 对 l = 1..L:
    2.1 Z^[l] = W^[l] @ A^[l-1] + b^[l]      // 线性变换（广播加偏置）
    2.2 A^[l] = g^[l](Z^[l])                 // 逐元素非线性
3. 返回 A^[1], ..., A^[L]
```

【算法逻辑解说】 1. **Step 2.1**：`W^[l] @ A^[l-1]` 是矩阵乘法——第 l 层的每个神经元对上一层所有输出做加权和。偏置 b 按列广播。 2. **Step 2.2**：激活逐元素施加。注意**必须在每层**施加非线性（除纯线性回归输出层）。 3. **缓存中间量**：前向传播中要**保存每层的** $Z^{[l]}, A^{[l]}$——反向传播（L12）需要它们计算梯度。这是“前向是后向的燃料”。 4. **维度检查**：$W^{[l]} \in \mathbb{R}^{n_l \times n_{l-1}}$、$Z^{[l]}, A^{[l]} \in \mathbb{R}^{n_l \times m}$——每步检查形状是调试神经网络的第一要务。 5. **数值稳定性**：Softmax + 交叉熵应使用 log-sum-exp 融合实现，避免中间指数溢出。

关键要点

1. 神经网络 = 分层复合“线性 + 非线性”；非线性激活是表达能力的来源。
2. 前向传播 = 从输入到输出的逐层变换；中间激活必须缓存供反向传播使用。
3. 输出层激活 + 损失函数 = 对标签分布的假设（GLM 原则的延伸）。
4. 向量化（矩阵乘法）与 mini-batch 是训练效率的根本。
5. 容量（宽/深）是过拟合风险源——正则化手段随后登场。

常见误区与注意事项

- **输出层误用 Sigmoid 做多分类**：多分类要用 Softmax（输出为合法概率分布）；Sigmoid 是逐元素的，不保证和为 1。
- **回归问题输出层加 Sigmoid/ReLU**：回归要求输出任意实数——输出层应为线性激活。
- **初始化全零权重**：对称性导致所有神经元学到相同特征（“对称破缺”失败）——需随机初始化（如 He/Xavier）。
- **ReLU 死亡**：学习率过大或初始化不当，神经元输出恒为负 → 梯度恒 0 → 永不复活。用 Leaky ReLU 或合理初始化/学习率。
- **忽略输入标准化**：大数值输入使 z 进入激活饱和区、梯度消失——标准化输入（均值 0 方差 1）是标配。

思考题

1. **问题**：为什么两层线性激活的网络等价于单层线性模型?
   - **答案**：$a^{[2]} = W^{[2]}(W^{[1]}x + b^{[1]}) + b^{[2]} = (W^{[2]}W^{[1]})x + (W^{[2]}b^{[1]} + b^{[2]})$——仍为 $W'x + b'$ 形式，无表达力增益。只有非线性激活才能让深度有意义。
2. **问题**：通用逼近定理说单隐层网络可逼近任意连续函数，那为什么还要深度?
   - **答案**：宽度大但浅的网络参数效率低（需要指数级神经元逼近某些函数）；深度网络通过**层级特征复用**以多项式级参数表达复杂函数，且泛化更好（结构先验：组合性）。实践中深而窄通常优于浅而巨宽。
3. **问题**：为什么 tanh 通常优于 Sigmoid 作隐藏层激活（除了梯度消失更缓）？
   - **答案**：tanh 输出零中心 $(-1, 1)$——后一层输入的均值接近 0，梯度更新更对称、收敛更快；Sigmoid 输出恒正 $(0, 1)$，导致权重梯度方向一致（全正/全负），zigzag 式更新。

Lecture 12: Neural Networks 2（反向传播 Backpropagation）

概述

本讲给出训练神经网络的“发动机”——**反向传播 (Backpropagation)**：一种高效计算损失对每个参数梯度的算法。核心是**链式法则**的智能组织：从输出层向前逐层传播“误差信号”，每个参数的梯度 = 本层激活 × 上游误差。本讲还讨论训练动态（学习率、权重衰减、Dropout）与自动微分的关系。

核心概念与数学直觉

- **问题**：最小化损失 $J(W, b)$（如交叉熵 + 正则），用梯度下降 $\theta := \theta - \alpha \nabla_\theta J$ 需要所有参数梯度。朴素方法（对每个参数数值差分）代价 $O(\text{参数数})$ 次前向传播——不可行。**反向传播用一次前向 + 一次反向，$O(\text{参数数})$ 总代价得到全部梯度**。
- **链式法则的组织**：定义第 l 层的**误差项**（损失对 $z^{[l]}$ 的梯度）：

$$\delta^{[l]} = \frac{\partial J}{\partial z^{[l]}}$$

由链式法则：

$$\delta^{[l]} = (W^{[l+1]T}\delta^{[l+1]}) \odot g'^{[l]}(z^{[l]})$$

$$\frac{\partial J}{\partial W^{[l]}} = \delta^{[l]}a^{[l-1]T}, \qquad \frac{\partial J}{\partial b^{[l]}} = \delta^{[l]}$$

```
*   $\odot$：逐元素（Hadamard）乘积。
*   $g'^{[l]}(z^{[l]})$：激活函数的导数—Sigmoid 为 $g(1-g)$，tanh 为 $1-g^2$，ReLU 为 $\mathbf
*   *直观解释*：误差 $\delta^{[l+1]}$ 从上层**反向传播**回来（乘 $W^{[l+1]T}$—“谁的权重大，谁担的责
*   *输出层初始化*：$\delta^{[L]} = \frac{\partial J}{\partial z^{[L]}}$ 由损失与输出激活直接给出
```

- **梯度消失/爆炸**：深层反向传播中 δ 每层乘 W^T 与 g'——若谱范数 <1（或 Sigmoid 饱和），误差指数衰减（**消失**，浅层学不动）；若 >1，指数放大（**爆炸**）。
  - *缓解*：ReLU 族激活（$g' = 1$ 正区）、合理初始化（He/Xavier 按层宽缩放）、批归一化、残差连接（ResNet）、梯度裁剪（clip）。
- **训练动态与正则化**：
  - *学习率调度*：固定 → 衰减（step/exponential/cosine）→ 自适应（Adam、RMSProp 按参数自适应步长）。
  - *权重衰减 (Weight Decay)*：L_2 正则——对应 L5 的岭回归思想。
  - *Dropout*：训练时随机“关掉”一部分神经元（伯努利掩码）——相当于训练大量共享权重的子网络再集成，降方差。
  - *批归一化 (BatchNorm)*：对每层激活做标准化，稳定训练、允许更大学习率。
- **自动微分 (Auto-Differentiation)**：反向传播 = **反向模式自动微分**在图上的实例化。现代框架（PyTorch/TensorFlow）把计算图构建与梯度传播自动化——但你仍需要理解 backprop 以调试（梯度检查、理解梯度爆炸）。

算法伪代码与逻辑解说：反向传播

伪代码

```
输入：
    - mini-batch (X, y)，网络参数 {W^[l], b^[l]}
    - 损失函数 J，激活 g^[l]

输出：
    - 梯度 dW^[l] = ∂J/∂W^[l], db^[l] = ∂J/∂b^[l]

1. 前向传播（缓存每层 Z^[l], A^[l]）：
   A^[0] = X; 对 l=1..L: Z^[l]=W^[l]A^[l-1]+b^[l]; A^[l]=g^[l](Z^[l])
2. 输出层误差: δ^[L] = ∂J/∂Z^[L]        // 如 Softmax+CE: A^[L] - Y_onehot
3. 对 l = L, L-1, ..., 1（反向）：
    3.1 dW^[l] = (1/m) * δ^[l] @ A^[l-1]^T   // 本层误差 × 前层激活
    3.2 db^[l] = (1/m) * sum(δ^[l], axis=1)
    3.3 若 l > 1: δ^[l-1] = (W^[l]^T @ δ^[l]) ⊙ g'^{[l-1]}(Z^[l-1])   // 误差继续回传
4. 返回 dW, db（供优化器做参数更新）
```

【算法逻辑解说】 1. **Step 1 前向 + 缓存**：必须保存 $Z^{[l]}$（激活导数需要）与 $A^{[l-1]}$（权重梯度需要）——这就是“前向的缓存是反向的燃料”。 2. **Step 2 输出层误差**：$\delta^{[L]} = A^{[L]} - Y_{\text{onehot}}$ 是“预测 − 真值”——与逻辑回归梯度同构，说明深度网络最后一层就是多分类逻辑回归。 3. **Step 3 反向循环**：从 L 到 1 逐层计算。$dW^{[l]} = \delta^{[l]} a^{[l-1]T}$：梯度 = “本层误差信号”外积“前层激活”——误差大且激活强 ⇒ 梯度大。 4. **Step 3.3 误差回传**：$\delta^{[l-1]} = (W^{[l]T}\delta^{[l]}) \odot g'(z^{[l-1]})$：上层误差按权重“分账”回传，再被本层激活斜率调制。注意 $W^{[l]T}$ 转置实现“反向传播”的实质——**梯度流的方向与权重矩阵的转置对应**。 5. **复杂度**：一次前向 $O(\text{参数量})$ + 一次反向 $O(\text{参数量})$——比逐参数数值差分快 $O(\text{参数量})$ 倍。 6. **梯度检查 (Gradient Checking)**：用数值差分 $\frac{J(\theta+\epsilon)-J(\theta-\epsilon)}{2\epsilon}$ 与反向传播梯度对比（相对误差 $< 10^{-7}$ 量级）验证实现正确性——调试必备。

关键要点

1. 反向传播 = 链式法则的智能排序：一次反向算出全部参数梯度。
2. 误差信号 δ 逐层回传：$\delta^{[l]} = (W^{[l+1]T}\delta^{[l+1]}) \odot g'(z^{[l]})$。
3. 梯度消失/爆炸是深层的核心工程难题：ReLU、好初始化、BatchNorm、残差连接是解药。
4. 反向传播 = 反向模式自动微分；理解它才能调试现代框架。
5. 正则化（权重衰减、Dropout、早停）与优化器（Adam）共同决定训练效果。

常见误区与注意事项

- **忘记缓存 Z**：反向传播需要激活导数 $g'(Z^{[l]})$——不缓存则无法计算（或需重算，浪费）。
- **逐元素 vs 矩阵梯度混淆**：δ 是矩阵（$n_l \times m$）；dW 是 δ 与 $A^{[l-1]T}$ 的**矩阵乘法**（不是逐元素）。形状检查是 first-class 调试手段。
- **激活函数求导错误**：Sigmoid 导数是 $g(1-g)$（不是 g' 本身）；ReLU 导数是指示函数。
- **不验证梯度**：实现 backprop 后必须做梯度检查；数值与解析梯度不一致时，先查形状、再查求导公式。
- **学习率过大导致 NaN/爆炸**：梯度爆炸常见症状是 loss 变 NaN——用梯度裁剪（clip norm）或降低学习率；同时检查权重初始化尺度。

- **Dropout 忘关（推理时）**：推理必须关闭 Dropout（或按保留概率缩放），否则预测随机化。

思考题

1. **问题**：推导单隐层网络（输入 → 隐层 tanh → 输出 sigmoid 二分类交叉熵）的输出层与隐层 δ。
   - **答案**：输出层：$\delta^{[2]} = a^{[2]} - y$（Sigmoid+CE 的简化形式）。隐层：$\delta^{[1]} = (W^{[2]T}\delta^{[2]}) \odot (1 - (a^{[1]})^2)$（tanh 导数 $1 - \tanh^2$）。权重梯度：$dW^{[2]} = \delta^{[2]}a^{[1]T}$，$dW^{[1]} = \delta^{[1]}x^T$。
2. **问题**：为什么深层网络用 Sigmoid 隐藏层容易梯度消失，而 ReLU 缓解?
   - **答案**：Sigmoid 导数最大 0.25，且输入远离 0 时趋于 0——每层误差至少乘 ≤ 0.25，10 层后 $\leq 0.25^{10} \approx 10^{-6}$，浅层几乎无梯度。ReLU 正区导数恒为 1，误差可无损回传（只要神经元激活）；负区为 0 只会“选择性”阻断。
3. **问题**：反向传播与数值差分相比，为什么既快又准?
   - **答案**：数值差分对每个参数需一次前向（$O(P)$ 次前向，P 为参数数），且受浮点截断误差限制（ϵ 不能太小）。反向传播利用链式法则**复用共享中间梯度**，一次前向 + 一次反向完成全部梯度，精度达机器精度。数值差分只用于“梯度检查”验证正确性。

第二部分：学习理论、实践建议与现代主题

Lecture 13: ML Advice（调试机器学习系统）

概述

本讲是“工程师视角”的核心课：**当模型效果不好时，下一步该做什么？** 内容基于 Andrew Ng 的经典讲义《Advice for Applying Machine Learning》：系统化的诊断方法（偏差/方差诊断、误差分析、消融实验）、调试学习算法的高效顺序，以及超参数调优策略。目标是把“拍脑袋试模型”变成“有依据的工程迭代”。

核心概念与数学直觉

- **调试学习算法的通用思路**：模型不好时，可能的原因很多（特征不足、正则化过强、数据太少、bug...）。**不要凭直觉乱试**——用诊断手段定位错误来源，让每次改动都有依据。
  - *诊断 (Diagnostic)*：一种能告诉你“问题出在哪”的测试。好的诊断像医生的检查：先定位，再治疗。
- **偏差/方差诊断（核心工具）**：对比**训练误差**与**验证（dev）误差**：
训练误差	验证误差	诊断	对策
高	高	**高偏差（欠拟合）**	更多特征/更大模型/更少正则化；换更强模型
低	高	**高方差（过拟合）**	更多数据/正则化/更小模型；集成
低	低	良好（或数据泄漏，需警惕）	—
高	低	异常（几乎不可能；检查数据泄漏/划分错误）	—
  - *学习曲线 (Learning Curves)*：画误差 vs 训练集大小。
    - 高偏差：训练误差与验证误差都高且**几乎不随数据增加而下降**——加数据没用，应加容量。

        - 高方差：训练误差低、验证误差高，且随数据增加两者**逐渐靠近**——加数据有效。
    - *数学直觉*：偏差是模型族的系统误差（容量不足时加数据无助）；方差随数据量下降（$O(1/\sqrt{m})$ 量级）——但方差只在高方差场景才是主要矛盾。
- **误差分析 (Error Analysis)**：对验证集上分错的样本**人工检查**，统计错误类型分布（如垃圾邮件分类：误判类别、漏掉关键词、图像模糊…），按频率与影响排序修复。
    - *直觉*：把 100 个错例分成几类，看哪类最多——修最大的那块。**数据决定上限，模型只是逼近上限**：若人工都难分，别指望模型。
- **消融实验 (Ablation Studies)**：**逐个移除/替换组件**，观察性能变化，确定每个组件（特征、模块、技巧）的真实贡献。例：去掉特征 A 性能掉 3%、去掉正则掉 1% ⇒ A 重要。这是论文与工程中验证“什么起作用”的标准方法。
- **优先级排序（Ng 的经验法则）**：
    1. **先让简单基线跑通**（线性模型/简单逻辑回归），再逐步复杂化；
    2. 优先**检查数据**（标签错误、泄漏、分布漂移）——数据问题往往比模型问题更常见；
    3. 用诊断定位偏差 vs 方差，再对症下药；
    4. 调参用**对数刻度网格** + 交叉验证，而不是随机点试。
- **超参数调优**：学习率、正则化强度、树深、网络宽度等。策略：粗网格 → 细网格；小数据上快速实验 → 全量训练；记录每次实验（配置、指标）形成日志。
- **关于“更多数据”**：加数据永远有用吗？——高偏差时没用（模型容量是瓶颈）；高方差时有用。**数据增强**（图像旋转/裁剪、文本回译）可视为“免费的数据”。

算法伪代码与逻辑解说：系统化调试流程

伪代码

```
输入：
    – 训练集、验证集、测试集（已正确划分）
    – 当前模型与训练流程

输出：
    – 改进方向清单（依据诊断）

1. 建立基线：训练简单模型，记录 train/dev/test 误差
2. 诊断偏差/方差：
    2.1 若 train 误差高 → 高偏差 → 加容量（特征/模型复杂度）或降正则化，回到 2
    2.2 若 dev 误差远高于 train → 高方差 → 加数据/正则化/降容量，回到 2
3. 若偏差方差都合理但 dev 仍差 → 误差分析：
    3.1 人工检查 dev 错例，按错误类型分类统计
    3.2 优先修复占比最大的错误类型（数据/特征/预处理层面）
4. 消融验证：逐个移除组件，确认每个组件贡献
5. 网格搜索关键超参数（对数刻度），用 CV 选最优
6. 最终在 test 上评估一次
```

【算法逻辑解说】 1. **Step 1 基线优先**：没有基线的“改进”无法衡量。简单模型也提供 dev 误差下界参考。 2. **Step 2 偏差/方差分流**：这是整个流程的“分叉口”——方向错了，后面全白做。注意必须用 **dev 误差**而非 train 误差判断。 3. **Step 3 误差分析的人力价值**：计算机只能告诉你“错多少”，人工能告诉你“错在哪类”。对文本/图像任务，看错例往往立刻发现数据问题（标签错、重复、分布偏）。 4. **Step 5 系统性调参**：按对数刻度扫 α, λ 等（数量级差异远大于细粒度差异）；用 CV 防过拟合调参过程本身。 5. **纪律**：test 集只碰一次——所有决策基于 dev；最终报告基于 test。

关键要点

1. 先诊断后动手：偏差/方差表 + 学习曲线定位主要矛盾。
2. 高偏差→加容量；高方差→加数据/正则化。方向错则事倍功半。
3. 误差分析 + 消融实验是“数据驱动改进”的核心工具。
4. 简单基线 → 系统化迭代 → 网格调参 → 一次性 test 评估。
5. 数据质量（标签、泄漏、分布）常是比模型更重要的瓶颈。

常见误区与注意事项

- **用 test 集反复调参**：这是数据泄漏，最终性能虚高。dev 集才是调参场。
- **数据泄漏的隐蔽形式**：预处理（标准化、缺失值填充）必须只用训练集统计量拟合；特征中含未来信息（时间序列）；重复样本跨划分。泄漏症状：dev/test 异常好。
- **高偏差时盲目加数据**：浪费算力且无效——先确认瓶颈（学习曲线）再行动。
- **调参只看单次运行**：随机性（初始化、mini-batch 顺序）造成噪声——多次运行取均值，或至少记录方差。
- **忽略错误分析直接换模型**：换模型是最贵、收益最不确定的动作；先看数据。

思考题

1. **问题**：训练误差 0.02、验证误差 0.25。诊断是什么？列三个合理对策。
   - **答案**：高方差（过拟合）。对策：(a) 增加训练数据；(b) 增大正则化（λ、Dropout、权重衰减）；(c) 减小模型容量（特征选择、更小网络/更浅树）；(d) 集成。学习曲线若显示两误差随数据增多而靠近，则加数据最有效。
2. **问题**：训练误差 0.4、验证误差 0.42，增加数据后两者几乎不动。诊断与对策?
   - **答案**：高偏差（欠拟合）——模型容量不足，数据再多也学不动。对策：增加特征/多项式特征、换更强模型（更深网络、更复杂模型族）、降低正则化。
3. **问题**：为什么预处理统计量只能从训练集计算?
   - **答案**：若用全量数据（含验证/测试）计算均值/方差做标准化，验证/测试信息在训练前就已泄漏进特征——评估乐观。正确做法：只在训练集上 fit 标准化器，再 transform 所有划分。

第三部分：强化学习（Reinforcement Learning）

Lecture 14: Basic Concepts in RL. Value Iteration. Policy Iteration

概述

本讲建立强化学习的数学框架：**马尔可夫决策过程 (MDP)** 形式化“智能体–环境”交互，定义**价值函数**与**贝尔曼方程**刻画“长期回报”，并给出求解最优策略的两大经典算法：**值迭代 (Value Iteration)** 与**策略迭代 (Policy Iteration)**。核心直觉：把“序贯决策”转化为“在状态空间上求解不动点方程”。

核心概念与数学直觉

- **马尔可夫决策过程 (MDP)**：元组 $(S, A, P_{sa}, \gamma, R)$
  - S：**状态集合**（环境的可能配置，如机器人位置/朝向）。
  - A：**动作集合**（智能体可执行的行为）。
  - P_{sa}：**状态转移概率**——在状态 s 执行动作 a 后转移到各状态的概率分布。**马尔可夫性**：下一状态只依赖当前状态与动作（历史无关）。
  - $\gamma \in [0, 1)$：**折扣因子**——权衡“当下收益”与“未来收益”。γ 小=目光短浅；γ 大=看重长远。保证无限时域回报有限（几何级数收敛）。
  - R：**奖励函数** $R : S \times A \to \mathbb{R}$（或 $R : S \to \mathbb{R}$）。
  - *策略 (Policy)*：$\pi : S \to A$（确定性）或 $\pi(a|s)$（随机）——智能体的行为规则。
  - *直觉*：MDP 是“带奖励的马尔可夫链 + 可控动作”。RL 的任务 = 找策略 π 使**期望折扣累积回报**最大。
- **价值函数**：
  - *状态价值*：`$V^\pi(s) = E\left[ \sum_{t=0}^{\infty} \gamma^t R(s_t) \mid s_0 = s, \pi \right]$`——从状态 s 出发按策略 π 行动的期望折扣回报。
  - *状态–动作价值（Q 函数）*：`$Q^\pi(s, a) = E\left[ \sum_{t=0}^{\infty} \gamma^t R(s_t) \mid s_0 = s, a_0 = a, \pi \right]$`——先执行动作 a 再按 π 行动。
  - *最优价值*：$V^*(s) = \max_\pi V^\pi(s)$；最优策略 π^* 满足 $\pi^*(s) = \arg\max_a Q^*(s, a)$。
- **贝尔曼方程 (Bellman Equation)**：价值函数的**递归自洽**关系：

$$V^\pi(s) = R(s) + \gamma \sum_{s'} P_{s\pi(s)}(s') V^\pi(s')$$

$$V^*(s) = R(s) + \gamma \max_a \sum_{s'} P_{sa}(s') V^*(s')$$

```
*   *直觉*：“从 $s$ 出发的价值 = 立即奖励 + 折扣后的未来价值期望”。贝尔曼方程把无限和化为**一步 + 递归**
*   *不动点视角*：$V^*$ 是贝尔曼最优算子的不动点；值迭代就是在反复应用该算子。
```

- **值迭代 (Value Iteration)**：`$V(s) := R(s) + \gamma \max_a \sum_{s'} P_{sa}(s') V(s')$`，重复直到收敛。

- 收敛后最优策略：$\pi^*(s) = \arg\max_a \sum_{s'} P_{sa}(s')V^*(s')$。
- *直觉*：从“只有 1 步视野”的价值开始，每轮迭代把视野**多往后看一步**（动态规划/自举）——价值逐渐向 V^* 收敛（折扣 $\gamma < 1$ 保证收敛）。

- **策略迭代 (Policy Iteration)**：交替两阶段：
  1. **策略评估 (Policy Evaluation)**：固定 π，解贝尔曼方程求 V^π（线性方程组，或迭代求解）。
  2. **策略改进 (Policy Improvement)**：$\pi'(s) = \arg\max_a \sum_{s'} P_{sa}(s')V^\pi(s')$——贪心改进，**保证不降**（策略改进定理）。
  3. 收敛：有限 MDP 中策略迭代在有限步收敛到 π^*（策略空间有限，单调改进）。
  4. *对比*：值迭代每步直接更新价值（隐含策略改进）；策略迭代显式维护策略。值迭代实现简单、实践中常用；策略迭代在状态数少时收敛更快（每步更贵）。

算法伪代码与逻辑解说：值迭代 / 策略迭代

伪代码 A：值迭代 (Value Iteration)

```
输入:
    - MDP (S, A, P_sa, γ, R), 收敛阈值 epsilon

输出:
    - 最优价值 V*, 最优策略 π*

1. 初始化 V(s) = 0 (所有 s)
2. 循环直到 max_s |V_new(s) - V(s)| < epsilon:
    2.1 对每个状态 s:
        V_new(s) = R(s) + γ * max_a Σ_{s'} P_sa(s') V(s')
    2.2 V = V_new
3. 对每个 s: π*(s) = argmax_a Σ_{s'} P_sa(s') V*(s')
4. 返回 V*, π*
```

伪代码 B：策略迭代 (Policy Iteration)

```
输入: MDP (S, A, P_sa, γ, R)
输出: 最优策略 π*

1. 初始化 π(s) = 任意动作 (所有 s)
2. 循环直到策略不再变化:
    // 策略评估: 解线性方程组 V^π = R + γ P_π V^π
    2.1 对每个 s: V^π(s) = R(s) + γ Σ_{s'} P_{sπ(s)}(s') V^π(s')
          (或用迭代法重复应用上式直至收敛)
    // 策略改进: 贪心
    2.2 对每个 s: π'(s) = argmax_a Σ_{s'} P_sa(s') V^π(s')
    2.3 若 π' == π: 终止; 否则 π = π'
3. 返回 π*
```

【算法逻辑解说】 1. **值迭代 Step 2**：每轮对每个状态做“一步展望”：当前奖励 + 最优后续价值。这是**动态规划**——利用子问题（后续状态的价值）的最优解构造当前最优解。γ 折扣保证映射是压缩映射 ⇒ 收敛唯一

不动点 V^*。 2. **值迭代 Step 3**：价值收敛后，策略 = 每状态选“能导向最高价值”的动作——注意**价值决定策略**，无需显式存储策略。 3. **策略迭代 Step 2.1**：固定策略下贝尔曼方程是**线性方程组**（$V = R + \gamma P_\pi V$），可解闭式（$(I - \gamma P_\pi)^{-1} R$）或迭代；状态数大时用迭代（每次 $O(|S|^2)$ 或稀疏加速）。 4. **策略改进定理**：π' 的贪心选择保证 $V^{\pi'} \geq V^\pi$ 逐状态成立——单调改进 + 策略有限 ⇒ 有限步收敛。 5. **前提（Model-based）**：两算法都需要已知 P_{sa}（模型）。当模型未知时，需要 L15 的**无模型/学习模型**方法。

关键要点

1. MDP 五元组 $(S, A, P_{sa}, \gamma, R)$：马尔可夫性 + 折扣回报是形式化的核心。
2. 贝尔曼方程 = 价值函数的递归自洽；V^* 是不动点。
3. 值迭代：直接逼近 V^*；策略迭代：评估–改进交替，单调收敛。
4. 两者都是**基于模型**的动态规划；模型未知时转向 L15。
5. 价值函数把“长期回报”压缩为单状态标量——序贯决策简化为逐状态贪心。

常见误区与注意事项

- **混淆 V 与 Q**：$V(s)$ 是“状态的价值”（隐含策略）；$Q(s,a)$ 是“状态–动作的价值”，策略选择用 Q（$\arg\max_a Q(s,a)$）。$V^*(s) = \max_a Q^*(s,a)$。
- **$\gamma = 1$ 不收敛**：无限时域无折扣时回报可能发散——$\gamma < 1$ 是收敛的数学保障（压缩映射）。
- **值迭代收敛判据**：用价值变化量（$\max_s |\Delta V| < \epsilon$）而非策略是否变化——价值接近最优时策略可能已最优但价值仍微调。
- **把奖励与回报混淆**：奖励 R 是立即信号；价值 V 是折扣累积回报——策略优化的是后者。
- **状态数爆炸（维度灾难）**：表格式 V 对每个状态存一个值，状态空间大（连续/高维）时不可行——引出函数近似（L15）。

思考题

1. **问题**：写出 $Q^*(s,a)$ 的贝尔曼方程，并说明它与 V^* 方程的关系。
   - **答案**：$Q^*(s,a) = R(s,a) + \gamma \sum_{s'} P_{sa}(s') \max_{a'} Q^*(s',a')$。关系：$V^*(s) = \max_a Q^*(s,a)$——把 Q^* 中的 max 代入即得 V^* 方程；反之 $Q^*(s,a) = R(s,a) + \gamma \sum_{s'} P_{sa}(s') V^*(s')$。
2. **问题**：值迭代第 k 轮后 V_k 的物理解释是什么？
   - **答案**：$V_k(s)$ = 从 s 出发、**至多 k 步**的最优期望折扣回报（之后截断）。每轮把视野延长一步；$k \to \infty$ 时 $V_k \to V^*$（几何收敛，误差 $O(\gamma^k)$）。因此值迭代天然可随时截断——实践中常用有限轮数近似。
3. **问题**：策略迭代中为什么“策略改进”保证不使性能变差？
   - **答案**：策略改进定理：对任意策略 π，定义 $\pi'(s) = \arg\max_a Q^\pi(s,a)$，则 $V^{\pi'}(s) \geq V^\pi(s)$ 对所有 s 成立。直觉：$Q^\pi(s, \pi'(s)) \geq Q^\pi(s, \pi(s)) = V^\pi(s)$——每状态选择当前最优动作，价值单调上升；等号仅在 π 已最优时成立。

Lecture 15: Model-based RL. Value Function Approximation

概述

上一讲假设转移概率 P_{sa} 已知（动态规划）。现实往往**模型未知**。本讲给出两条路线：**模型学习**（从数据估计 P_{sa} 再套用 DP）与**无模型学习**（直接学习价值函数/策略）。重点算法：**Q-learning**（无模型、off-policy 的价值迭代近似）、**拟合值迭代 (Fitted Value Iteration)**（价值函数的函数近似，处理连续状态），以及 REINFORCE（策略梯度，处理连续/随机动作）。

核心概念与数学直觉

- **两条路线总览**：
路线	思想	代表算法
Model-based	先学模型（$\hat{P}_{sa}$、$\hat{R}$），再在模型上做规划	估计转移概率 + 值迭代；模拟/规划
Model-free	跳过模型，直接从经验学价值/策略	Q-learning、SARSA、策略梯度
  - *直觉*：Model-based 像“先画地图再找路”；Model-free 像“不画地图，边走边记哪条路好”。
- **Q-learning（无模型值迭代）**：

$$Q(s,a) := Q(s,a) + \alpha \left[R(s) + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

```
*   $\alpha$: 学习率（步长）。
*   $s'$: 执行 $a$ 后**实际观察到的**下一状态（不需要 $P_{sa}$! ）。
*   $R(s) + \gamma \max_{a'} Q(s',a')$: **TD 目标 (temporal difference target)**——用一步经验 +
*   $R(s) + \gamma \max_{a'} Q(s',a') - Q(s,a)$: **TD 误差**——预测与目标的差距。
*   *off-policy 特性*: 更新用的 $\max_{a'} Q(s',a')$ 与**实际采取的动作无关**——可以用任意行为策略收集
*   *收敛条件*: 每个 $(s,a)$ 被无限次访问 + 学习率满足 Robbins-Monro 条件（$\sum \alpha_t = \infty
```

- **连续状态：价值函数近似 (Value Function Approximation)**：状态空间连续/巨大时无法查表。用参数化函数 $V_\theta(s)$（如线性、神经网络）近似 $V(s)$。
  - *拟合值迭代 (Fitted Value Iteration)*：每轮用当前 $\hat{V}$ 构造回归目标 $y^{(i)} = R(s^{(i)}) + \gamma \max_a \hat{P}_{sa}$ 的期望（或从经验估计），再监督学习拟合 $V_\theta \approx y$——把 DP 变成**回归问题**。
  - *线性函数近似*：$V_\theta(s) = \theta^T \phi(s)$（$\phi$ 为状态特征）——最小二乘拟合目标值；配 Q-learning 即线性 Q-learning（如 DQN 的线性前身）。
  - *深度 Q 网络 (DQN)*：神经网络 Q_θ + 经验回放 + 目标网络稳定训练——AlphaGo、Atari 的核心组件（本课以概念为主）。
- **策略搜索与 REINFORCE**：
  - *思想*：不再学价值，直接参数化策略 π_θ（如“状态 → 动作分布”的神经网络），用梯度上升最大化期望回报。
  - *问题*：期望回报对 θ 的梯度涉及未知环境——**似然比技巧 (likelihood ratio trick)**：`$\nabla_\theta E_\tau[R(\tau)] = E_\tau\left[ R(\tau) \nabla_\theta \log p_\theta(\tau) \right]$`，其中 $p_\theta(\tau) = p(s_0) \prod_t \pi_\theta(a_t|s_t) p(s_{t+1}|s_t, a_t)$——环境动力学 $p(s'|s,a)$ 不依赖 θ，梯度中自动消失！

- *REINFORCE 更新*：`$\theta := \theta + \alpha \sum_{t} \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot R_t$`（R_t 为从 t 起的折扣回报）
  - *直觉*：采样若干轨迹；回报高的轨迹上的动作概率被**推高**（“好轨迹的动作更可能被重复”），回报低的被压低。这是“试错 + 强化”的直接实现。
- *方差问题*：采样回报方差大 → 用**基线 (baseline)**（减均值）与**Actor–Critic**（用价值函数代替整条回报）降方差。

算法伪代码与逻辑解说：Q–learning（表格版）

伪代码

```
输入:
    – 环境（可交互采样）: 状态 s，动作 a，奖励 r，下一状态 s'
    – 学习率 alpha，折扣 gamma，探索率 epsilon，回合数 N

输出:
    – Q 表 Q(s, a)

1. 初始化 Q(s, a) = 0（所有 s, a）
2. 对回合 ep = 1..N:
    2.1 重置环境，得到初始状态 s
    2.2 循环直到回合结束:
        2.2.1 选动作: 以概率 epsilon 随机探索，否则 a = argmax_a' Q(s, a')  // ε–greedy
        2.2.2 执行 a，观察 r 与 s'
        2.2.3 更新: Q(s, a) += alpha * (r + gamma * max_a' Q(s', a') – Q(s, a))
        2.2.4 s = s'
3. 返回 Q
```

【算法逻辑解说】 1. **Step 2.2.1 ε–greedy 探索**：以 ϵ 概率随机动作保证“每个 (s,a) 都被访问”——Q–learning 收敛的前提是充分探索。ϵ 常随时间衰减（先探索后利用）。 2. **Step 2.2.3 TD 更新**：**不需要模型**——s' 来自真实环境观察。$\max_{a'} Q(s',a')$ 使用**当前最优估计**（自举，bootstrap）：用估计更新估计，这正是 TD 方法“从猜测中学习”的本质。 3. **off–policy**：行为策略（ε–greedy）与目标策略（贪心）不同——Q 最终逼近**贪心最优策略**的 Q^*。 4. **对比蒙特卡洛**：MC 用整条轨迹的回报更新（高方差、无偏）；TD 用一步更新（低方差、有偏）。Q–learning 是 TD 家族成员。 5. **收敛**：理论上在表格 + 无限探索 + 合适 α 下收敛到 Q^*；实践中配合函数近似（DQN 等）处理大规模状态。

算法伪代码与逻辑解说：REINFORCE（策略梯度）

伪代码

```
输入:
    - 参数化策略 π_θ(a|s)，学习率 alpha，回合数 N

输出:
    - 策略参数 θ

1. 初始化 θ
2. 对回合 ep = 1..N:
    2.1 采样一条轨迹: τ = (s_0, a_0, r_1, s_1, a_1, r_2, ..., s_T)
        (每一步 a_t ~ π_θ(·|s_t))
    2.2 计算每步折扣回报: R_t = Σ_{k=t..T} γ^(k-t) r_k
    2.3 累计梯度: g = Σ_t R_t * ∇_θ log π_θ(a_t | s_t)
    2.4 更新: θ = θ + alpha * g          // 梯度上升（最大化期望回报）
3. 返回 θ
```

【算法逻辑解说】 1. **Step 2.1 采样**：策略是随机的（输出动作分布），从分布中采样动作——探索内建于策略。 2. **Step 2.2 回报计算**：R_t 是“事后诸葛亮”——用整条轨迹的真实回报评价动作。 3. **Step 2.3 核心公式**：$\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot R_t$——**动作概率的梯度 × 回报**。回报正 → 概率上升方向；负 → 下降方向。直觉：像“体罚/奖赏”调整每个动作的倾向。似然比技巧使环境动力学（未知）从梯度中消失——只需知道自己的策略。 4. **Step 2.4**：梯度上升（不是下降——目标是最大化回报）。可用基线减方差：$\theta := \theta + \alpha \sum_t \nabla \log \pi_\theta \cdot (R_t - b)$。 5. **高方差警告**：整条轨迹回报方差大——需要大量采样；Actor–Critic 用价值函数做基线/替代回报，是现代（PPO/A2C）的基础。

关键要点

1. 模型未知时：Model–based（先学模型）或 Model–free（直接学价值/策略）。
2. Q–learning：无模型 TD 更新，off–policy，ϵ–greedy 保证探索。
3. 连续状态 → 价值函数近似（拟合值迭代、DQN）——把 RL 转成回归问题。
4. REINFORCE：策略梯度 + 似然比技巧，直接优化策略；高方差需基线/Actor–Critic。
5. 探索–利用权衡是 RL 的核心张力：ϵ 衰减、策略随机性是常用机制。

常见误区与注意事项

- **混淆 on–policy 与 off–policy**：Q–learning 是 off–policy（可复用历史/他人经验）；SARSA 与策略梯度是 on–policy（数据必须来自当前策略）。用错会学到错误目标。
- **Q–learning 更新漏掉 γ 或 max**：TD 目标 $r + \gamma \max_{a'} Q(s', a')$ 三项缺一不可；漏 $\max$ 退化为 SARSA 的目标。
- **ϵ 恒为 0（纯利用）**：从不探索 → 学到的 Q 只反映已访问路径，可能远非最优。
- **REINFORCE 忘记基线**：裸 REINFORCE 方差巨大，训练不稳定；基线（如状态价值）几乎总是必要。
- **奖励设计不合理**：奖励稀疏（只在终点给）→ 学习极慢；奖励过于密集/易被钻空子 → 学到投机行为（reward hacking）。奖励设计是 RL 工程的核心。
- **函数近似 + 自举的稳定性**：Q 网络 + TD 自举 + 非线性近似三者叠加易发散——经验回放与目标网络（DQN 技巧）缓解。

思考题

1. **问题**：Q-learning 为什么被称为 off-policy？SARSA 的更新是什么、为什么是 on-policy？
   - **答案**：Q-learning 更新用 $\max_{a'} Q(s', a')$（目标策略是贪心），与行为策略（如 ε-greedy）无关 ⇒ off-policy。SARSA 更新用 $Q(s', a')$（a' 是行为策略实际要执行的动作）⇒ 学的是**当前行为策略**的价值 ⇒ on-policy。on-policy 更稳（学啥用啥），off-policy 更省数据（可复用）。
2. **问题**：拟合值迭代如何把“无模型 + 连续状态”问题转化为回归问题？
   - **答案**：采样大量 $(s^{(i)}, a^{(i)}, r^{(i)}, s'^{(i)})$ 经验；构造回归目标 $y^{(i)} = r^{(i)} + \gamma \max_a \hat{Q}(s'^{(i)}, a)$（用当前近似 $\hat{Q}$ 计算）；用监督学习（最小二乘/神经网络）拟合 $Q_\theta \approx y$；重复多轮。价值近似把 RL 变成“不断构造数据集 + 拟合”的循环——这是 DQN 的骨架。
3. **问题**：推导 REINFORCE 中 $\nabla_\theta \log p_\theta(\tau)$ 为何不含环境动力学。
   - **答案**：$p_\theta(\tau) = p(s_0) \prod_t \pi_\theta(a_t|s_t) p(s_{t+1}|s_t, a_t)$。取 $\log$ 后对 θ 求导：$p(s_0)$ 与 $p(s_{t+1}|s_t, a_t)$ 均不含 θ，导数全为 0——只剩 $\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)$。这就是“策略梯度只需知道自己的策略、无需环境模型”的数学依据。

第四部分：无监督学习进阶与降维

Lecture 16: Principal Components Analysis (PCA)

概述

本讲回到无监督学习，解决**降维**问题：高维数据冗余多、可视化难、计算慢。**主成分分析 (PCA)** 找到数据方差最大的若干正交方向（主成分），把数据投影到低维子空间，同时**尽可能保留数据的变化信息**。本讲还简述其姊妹方法 **ICA**（独立成分分析，解决“鸡尾酒会问题”）。PCA 是数据预处理、可视化、去噪的标配工具。

核心概念与数学直觉

- **问题定义**：数据 $\{x^{(i)}\}_{i=1}^m$，$x^{(i)} \in \mathbb{R}^d$，希望投影到 k 维（$k \ll d$）子空间，使投影后的数据**方差最大**（信息损失最小）。
  - *直观解释*：数据点云像一个“扁椭球”——PCA 找到椭球的**主轴**（方差最大的方向）。降维 = 沿主轴切开投影，丢掉方差最小的方向（视为噪声/冗余）。
  - *例子*：汽车数据中“最高时速(mph)”与“最高时速(km/h)”几乎线性相关——PCA 自动发现主轴方向，把两个冗余特征合成一个。
- **算法步骤（数据预处理 → 协方差 → 特征分解）**：
  1. **标准化**：$\tilde{x}^{(i)} = x^{(i)} - \frac{1}{m} \sum_j x^{(j)}$（去均值；可选除以标准差归一化量纲）。
  2. **协方差矩阵**：`$\Sigma = \frac{1}{m} \sum_{i=1}^{m} \tilde{x}^{(i)} \tilde{x}^{(i)T} = \frac{1}{m} X^T X$`（X 为去均值后的 $m \times d$ 矩阵）。
     - Σ 是**对称半正定**矩阵——其特征向量正交、特征值非负。

3. **特征分解**：求 Σ 的特征向量（按特征值降序 $\boldsymbol{u_1, \ldots, u_d}$）。
   - 特征值 $\boldsymbol{\lambda_j}$ = 数据沿特征向量 $\boldsymbol{u_j}$ 方向的**方差**。
4. **投影**：取前 $\boldsymbol{k}$ 个特征向量组成 $\boldsymbol{U_k = [u_1 \ldots u_k] \in \mathbb{R}^{d \times k}}$，新表示：`$z^{(i)} = U_k^T \tilde{x}^{(i)} \in \mathbb{R}^{k}$`（把 $\boldsymbol{d}$ 维投影到 $\boldsymbol{k}$ 维）。
5. *重构*：$\boldsymbol{\hat{x}^{(i)} = U_k z^{(i)} + \bar{x}}$——PCA 是最小化重构误差（$\sum_i \|x^{(i)} - \hat{x}^{(i)}\|^2$）的线性投影，与“最大方差”是同一枚硬币的两面。
6. *解释方差比例*：前 $\boldsymbol{k}$ 个主成分保留的信息量 = $\frac{\sum_{j=1}^{k} \lambda_j}{\sum_{j=1}^{d} \lambda_j}$——常用于选 $\boldsymbol{k}$（如保留 95% 方差）。

- **PCA 与相关方法的联系**：
  - **与 SVD 的关系**：$\boldsymbol{X = U\Sigma V^T}$，$\boldsymbol{X^T X}$ 的特征向量 = $\boldsymbol{V}$ 的列——SVD 数值上更稳定，是 PCA 的标准实现（`np.linalg.svd`）。
  - **与因子分析 (Factor Analysis, notes9) 的区别**：因子分析是**概率模型**（$\boldsymbol{x = \Lambda z + \mu + \epsilon}$，用 EM 估计）；PCA 是**直接代数方法**（特征分解）。因子分析假设显式噪声结构，PCA 假设“丢弃方向是噪声”。
  - **ICA (独立成分分析)**：$\boldsymbol{x = As}$，$\boldsymbol{s}$ 的分量**统计独立**（非高斯）。目标找 $\boldsymbol{W = A^{-1}}$ 使 $\boldsymbol{Wx}$ 分量独立。与 PCA（去相关，二阶矩）不同，ICA 利用**高阶矩/非高斯性**（如最大化峰度或负熵）。典型应用：鸡尾酒会问题（多麦克风分离多说话人）。
- **PCA 的用途与陷阱**：
  - 可视化（降到 2/3 维）、去相关（利于后续线性模型）、去噪、压缩、加速训练。
  - **陷阱**：PCA 是无监督的——它不在乎标签；用 PCA 降维后分类可能**变差**（丢掉的维度可能含判别信息）。不应盲目用 PCA 防止过拟合（正则化/更多数据通常更有效）。

算法伪代码与逻辑解说：PCA

伪代码

```
输入:
    – 数据矩阵 X（m×d），目标维度 k（或保留方差比例 p）

输出:
    – 投影矩阵 U_k（d×k），均值 mu，降维后的数据 Z（m×k）

1. 去均值: mu = (1/m) * Σ_i x^(i);  X_c = X − mu
   （可选）按特征标准差归一化: X_c /= std(X_c)
2. 协方差: Sigma = (1/m) * X_c^T @ X_c          // d×d
3. 特征分解: [U, S, V] = svd(Sigma) 或 eig(Sigma)
   // 特征向量按特征值（=方差）降序排列
4. 选 k:
   – 若给定 k: U_k = U[:, :k]
   – 若给定保留比例 p: 取最小 k 使 Σ_{j=1..k} S_j / Σ_j S_j >= p
5. 投影: Z = X_c @ U_k                           // m×k
6. 返回 U_k, mu, Z
```

【算法逻辑解说】 1. **Step 1 标准化**：PCA 对量纲敏感——身高(cm)与体重(kg)混合时，方差大的特征主导主轴。先减去均值（协方差的中心化）；若要各特征等权，再除以标准差（用相关矩阵替代协方差矩阵）。 2. **Step 2 协方差**：$\Sigma_{jl} = \frac{1}{m}\sum_i \tilde{x}_j^{(i)}\tilde{x}_l^{(i)}$——度量特征 j 与 l 的**共变**程度。PCA 的本质是“对角化这个共变矩阵”。 3. **Step 3 特征分解**：特征向量 u_j 是数据的主轴方向；特征值 λ_j 是沿该轴的方差。用 SVD 数值更稳（避免显式构造 X^TX 的平方放大）。 4. **Step 4 选 k**：保留方差比例是客观准则（如 95%）；或按下游任务（可视化用 2/3）选择。 5. **Step 5 投影**：$Z = X_c U_k$——每行是样本在前 k 个主成分上的坐标（线性组合系数）。 6. **复杂度**：SVD 约 $O(\min(md^2, m^2d))$——特征多时先考虑随机化 SVD 或增量 PCA。

关键要点

1. PCA = 找数据方差最大的正交方向（主轴），投影到低维子空间，最大化保留信息。
2. 数学核心：协方差矩阵的特征分解；特征值 = 各方向方差。
3. “最大方差”与“最小重构误差”等价——PCA 是最优的线性降维（对给定 k）。
4. PCA 是无监督、线性、二阶统计（去相关）；ICA 是线性但利用高阶统计（独立性）。
5. 用途：可视化、去噪、压缩、加速；**不要**把 PCA 当万能防过拟合工具。

常见误区与注意事项

- **忘记标准化**：不同量纲特征直接算协方差 → 大数值特征霸占主轴。先标准化（尤其特征单位不同时）。
- **把 PCA 当特征选择**：PCA 生成的是**线性组合**（所有原始特征的加权和），不是选子集——可解释性更差。
- **认为 PCA 总能提升分类**：PCA 不考虑标签，可能丢弃判别信息；分类前应比较“原始特征 vs PCA 特征”。
- **选 k 只看经验**：用“保留 95% 方差”客观准则或下游任务验证，而非拍脑袋。
- **对异常值敏感**：PCA 基于二阶矩，对离群点极敏感——先清洗/鲁棒化（如中位数中心化）。
- **混淆 PCA 与 ICA 目标**：PCA 最大化方差（去相关，二阶矩）；ICA 最大化独立性（非高斯性，高阶矩）——鸡尾酒会问题必须用 ICA 类方法。

思考题

1. **问题**：证明“最大化投影方差”与“最小化重构误差”对 PCA 是等价的。
   - **答案**：设投影方向 u（单位向量），样本 $\tilde{x}$ 投影 $z = u^T\tilde{x}$，重构 $\hat{x} = zu$。重构误差 $\|\tilde{x} - \hat{x}\|^2 = \|\tilde{x}\|^2 - (u^T\tilde{x})^2$。对所有样本求和：$\sum_i \|\tilde{x}^{(i)}\|^2$ 是常数，故最小化重构误差 $\iff$ 最大化 $\sum_i (u^T\tilde{x}^{(i)})^2 = u^T(\sum_i \tilde{x}^{(i)}\tilde{x}^{(i)T})u = m \cdot u^T\Sigma u$——即最大化沿 u 的方差。约束 $\|u\| = 1$ 下，解为 Σ 最大特征值对应特征向量。
2. **问题**：数据是 3 维且三个特征完全线性相关（秩 1），PCA 的 $k = 1$ 能保留多少方差？为什么？
   - **答案**：100%。协方差矩阵秩 1 ⇒ 只有一个非零特征值——数据全部落在一条直线上（一维子空间）。PCA 的 $k = 1$ 主成分完全重构数据（零重构误差）。这正是“冗余特征可被自动消除”的极端例证。
3. **问题**：为什么 ICA 无法用协方差（二阶矩）解决？PCA 预白化对 ICA 有何帮助？

- **答案**：若 s 各分量独立，则 $x = As$ 的协方差 $\Sigma_x = A\Sigma_s A^T$——任意正交旋转 A 都给出相同的去相关结果，二阶矩无法区分旋转。ICA 需高阶统计（非高斯性）确定唯一解。PCA 白化（$\Sigma_x^{-1/2}$）先把数据去相关、归一化，将问题化简为“在正交变换中找独立性”——是标准 ICA 预处理步骤。

第五部分：现代主题——大语言模型

Lecture 17: Large Language Models — Learning Tasks, Language Modeling, Embeddings, Transformers

概述

本讲把课程知识（监督学习、GLM、神经网络、自监督学习）汇聚到现代深度学习的旗舰：**大语言模型 (LLM)**。核心问题：模型如何“学会”语言？答案是**语言建模**——在大规模文本上做自监督的“预测下一个词”，由此涌现出理解、推理、生成能力。本讲覆盖词嵌入、注意力机制与 Transformer 架构，以及 LLM 的训练范式（预训练 → 对齐）。

核心概念与数学直觉

- **语言建模 (Language Modeling)**：给定前文 $x_1, \ldots, x_{t-1}$，预测下一个词的概率分布 $P(x_t | x_1, \ldots, x_{t-1})$。
  - *自监督视角*：文本本身就是标签（下一个词）——无需人工标注，数据近乎无限。这是 LLM 规模化的基石。
  - *损失*：交叉熵（负对数似然）：`$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T} \log P_\theta(x_t | x_{<t})$`——最小化平均预测困惑度。
  - *困惑度 (Perplexity)*：$\mathrm{PPL} = \exp(\mathcal{L})$——模型对下一个词的“平均困惑程度”，越低越好。
  - *训练范式*：**预训练 (pretraining)**（海量文本自监督）→ **指令微调 (instruction tuning)**（人类指令 + 答案监督）→ **对齐 (alignment)**（RLHF/DPO 让输出符合人类偏好）。
- **词嵌入 (Word Embeddings)**：
  - *问题*：词是离散符号（one-hot 是 V 维、无语义结构）。嵌入 = 把每个词映射为稠密向量 $e_w \in \mathbb{R}^d$，使**语义相近的词向量相近**。
  - *直觉*：`king − man + woman ≈ queen`——嵌入空间编码语义关系（类比推理）。向量空间里，“性别”是 king→queen 的方向，“皇权”是 king→queen 的另一维度。
  - *从 Word2Vec 到上下文嵌入*：静态嵌入（Word2Vec/GloVe，每个词一个向量，无法区分多义词）→ **上下文嵌入**（如 BERT/LLM 内部表示：同一个词在不同句子的向量不同，由其上下文决定）。
  - *现代视角*：LLM 第一层把 token 映射到嵌入，各层 Transformer 不断“精炼”这些表示——最后层输出即上下文相关的语义向量。
- **注意力机制 (Attention)**：让模型在预测每个词时**有选择地关注输入中的相关部分**。

- *核心公式（缩放点积注意力）*：

$$\text{Attention}(Q,K,V)=\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

```
    *   $Q$（Query，查询）：“我在找什么”——当前词的表示。
    *   $K$（Key，键）：“我是什么”——各输入词的标签。
    *   $V$（Value，值）：“我提供什么”——各输入词的内容。
    *   $QK^T$：查询与所有键的点积 = 相似度分数；除以 $\sqrt{d_k}$ 防点积过大使 softmax 饱和（梯度消失
    *   softmax：把分数变成注意力权重（和为 1 的分布）。
    *   $\times V$：按权重**加权求和**各词内容——输出是“输入中与我相关的信息的混合”。
*   *直觉类比*：阅读理解时，读到“它”会去文中找“它指代谁”（Query 匹配 Key），然后把那个词的信息（Value）带
*   *多头注意力（Multi-head）*：并行多组 $(Q,K,V)$ 投影，每组关注不同关系（语法、指代、语义），拼接后线性
```

- **Transformer 架构**（GPT/BERT 的骨干）：
  - *结构*：嵌入层 → L 层 Transformer 块（每块 = 多头自注意力 + 前馈网络(MLP)，各带残差连接与 LayerNorm）→ 输出层。
  - *自注意力 (Self-attention)*：Q,K,V 都来自**同一序列**——每个位置关注序列内所有位置（含自身），建模**长距离依赖**（对比 RNN 的逐步传播，注意力是 $O(1)$ 路径直达任意远）。
  - *位置编码 (Positional Encoding)*：注意力本身**不感知顺序**（置换等变）——必须注入位置信息（正弦位置编码或可学习位置嵌入）。这是“词序信息从哪来”的答案。
  - *因果掩码 (Causal Masking)*：语言模型的自注意力只允许看**过去的词**（上三角掩码），保证预测 x_t 时看不到未来——训练与推理一致。
  - *前馈层 (MLP)*：逐位置的非线性变换——存储“知识/模式”（研究表明 MLP 承担大量事实记忆）。
  - *为什么深度 Transformer 有效*：注意力负责“信息路由”（谁关注谁），MLP 负责“信息加工”（把路由来的信息变换）；残差连接保证深层梯度流动（呼应 L12 梯度消失）。
- **LLM 的关键能力与规模化**：
  - *涌现能力 (Emergent abilities)*：模型规模（参数量、数据量）跨越阈值后，思维链、指令遵循、少样本学习等能力**突然出现**——并非显式设计。
  - *缩放定律 (Scaling Laws)*：损失随参数量、数据量、计算量的幂律下降——投入与收益可预测，驱动“越大越好”竞赛。
  - *训练三要素*：数据（质量/多样性）、规模（参数）、对齐（RLHF）。数据质量常比参数量更重要。

算法伪代码与逻辑解说：GPT 式自回归生成

伪代码

```
输入:
    - 预训练 LLM（Transformer 解码器）: P_θ(x_t | x_<t)
    - 提示词 prompt，生成长度 max_new_tokens，采样温度 T，top-p 阈值 p

输出:
    - 生成的续写文本

1. tokens = tokenize(prompt)                        // 词元化: 文本 → token id 序列
2. 对 step = 1..max_new_tokens:
    2.1 logits = model(tokens)                      // 前向: 最后位置输出词表 logits
    2.2 logits = logits[-1] / T                     // 温度缩放（T<1 更确定, T>1 更多样）
    2.3 probs = softmax(logits)
    2.4 若使用 top-p: 截断累积概率 <= p 的最小集合，重归一化
    2.5 next_token = sample(probs)                  // 从分布采样（或 argmax = 贪心）
    2.6 tokens.append(next_token)
3. 返回 detokenize(tokens)
```

【算法逻辑解说】 1. **Step 1 词元化 (Tokenization)**：文本切成子词单元（如 BPE）——词表约 3 万–20 万，平衡“生词覆盖”与“序列长度”。 2. **Step 2.1 前向**：解码器每步只取**最后一个位置**的 logits——但自注意力已把整个前缀的信息汇聚到该位置（KV 缓存可加速：只需算新 token 的注意力）。 3. **Step 2.2–2.4 采样控制**：温度 T 调节分布尖锐度（$T \to 0$ 贪心、$T \to \infty$ 均匀）；top-p 截断低概率尾巴防“胡言乱语”。**解码策略是生成质量的工程核心**。 4. **Step 2.5 自回归循环**：每次生成一个 token 并拼回输入——这就是“自回归 (autoregressive)”：预测下一步，把结果当输入再预测下一步（与马尔可夫链的迭代精神一致）。 5. **复杂度**：生成 N 个 token 需 N 次前向（序列长 $O(N)$）——总 $O(N^2)$ 量级（注意力二次复杂度），长文本生成慢是工程挑战（FlashAttention、KV 缓存缓解）。

关键要点

1. LLM 的核心任务是语言建模：预测下一个词的自监督学习——数据=文本本身。
2. 嵌入把离散词映射为语义向量空间（king−man+woman≈queen）。
3. 注意力 = 软检索：Q 匹配 K、按权重聚合 V；多头捕获多种关系。
4. Transformer = 注意力（信息路由）+ MLP（信息加工）+ 残差/归一化（稳定训练）；因果掩码保证自回归一致性。
5. 规模 + 数据 + 对齐 → 涌现能力；困惑度/交叉熵是核心训练指标。

常见误区与注意事项

- **混淆嵌入与上下文表示**：静态嵌入（每词一个向量）≠ LLM 逐层上下文表示（每词每层一个向量）——后者才编码语境。
- **忽视因果掩码**：训练时若不掩码未来 token，模型“偷看答案”，测试（自回归）时性能崩盘。
- **注意力二次复杂度**：长上下文（100k+ tokens）下朴素注意力内存爆炸——需稀疏注意力/FlashAttention 等工程手段。
- **用困惑度直接比较不同词表模型**：困惑度依赖词元化方式（BPE 粒度不同不可比）；同词表内比较才公平。

- **把“预测下一个词”误读为“死记硬背”**：语言建模的副产品（压缩=理解）使其获得推理能力；但幻觉（编造事实）仍是固有风险（L18 讨论缓解）。
- **忽略位置编码**：把句子打乱重排后注意力结果不变——位置信息必须显式注入，否则模型是“词袋 Transformer”。

思考题

1. **问题**：为什么注意力要除以 $\sqrt{d_k}$?
   - **答案**：Q, K 各分量若近似独立同分布（均值 0、方差 1），点积 $Q \cdot K$ 的方差为 d_k——维度越大点积越大，softmax 输入进入饱和区，梯度趋近 0。除以 $\sqrt{d_k}$ 把方差归一化回 1，保持 softmax 梯度健康。
2. **问题**：RNN 与 Transformer 在处理长序列时各自的瓶颈是什么?
   - **答案**：RNN 逐时间步传播隐藏状态——长距离信息要经过 $O(L)$ 步传递，梯度消失/爆炸严重（L12 同款问题）。Transformer 自注意力任意两个位置**一步直达**（$O(1)$ 路径），长距离依赖建模天然更优；但注意力对所有位置两两计算，$O(L^2)$ 时间/内存（长序列贵）。两者是“路径长度 vs 计算量”的权衡。
3. **问题**：温度 $T \to 0$ 与 $T \to \infty$ 的生成各有什么风险?
   - **答案**：$T \to 0$（贪心）输出确定、连贯但**重复乏味**（陷入高频模式）；$T \to \infty$（均匀采样）输出多样但**语无伦次**。实践中 $T \in [0.7, 1.0]$ + top-p 常用——多样性（创造）与一致性（正确）的权衡，类似探索-利用。

Lecture 18: Large Language Models — RAG, Fine-tuning, Prompt Optimization, Safety

概述

上一讲回答了“LLM 如何工作”；本讲回答工程问题：“**如何把 LLM 用于真实任务并控制风险**”。四大主题：**RAG**（外挂知识库缓解幻觉与时效）、**微调**（用任务数据调整模型）、**提示优化**（不改权重只改输入）、**安全**（对齐、越狱、隐私）。核心思想：**知识更新、行为定制、成本控制、风险治理**各有各的工具箱。

核心概念与数学直觉

- **RAG（检索增强生成, Retrieval-Augmented Generation）**：
  - *动机*：LLM 的知识截止于训练时刻；幻觉（编造事实）源于“凭记忆作答”。RAG = **先检索再生成**：把外部知识库作为“开卷考试”的参考书。
  - *流程*：查询 → 检索相关文档片段（向量相似度）→ 拼接进 prompt → LLM 基于上下文生成。
  - *向量检索*：文档切块 → 嵌入向量 → 建索引（FAISS）；查询嵌入后找**余弦相似度**最高的 k 个片段。余弦相似度：$\cos(u, v) = \frac{u \cdot v}{\|u\|\|v\|}$——只关心方向（语义）不关心长度。
  - *为什么有效*：把“事实记忆”外包给检索器，LLM 只需“阅读理解 + 组织语言”——**记忆与推理解耦**；幻觉率显著下降、知识可实时更新（更新文档库即可）。

- *局限*：检索质量决定上限（检索不到 → 生成没依据）；长文档切片策略、重排序（rerank）影响精度。

- **微调 (Fine-tuning)**：
  - *思想*：在预训练权重上继续训练，适配特定任务/风格/领域。
  - *全量微调*：更新全部参数——效果好但代价高（每任务一套模型）。
  - *参数高效微调 (PEFT)*：只更新少量参数。
    - LoRA：冻结原权重 $\boldsymbol{W}$，训练低秩增量 $\boldsymbol{W}+\boldsymbol{BA}$（$\boldsymbol{B}\in\mathbb{R}^{d\times r},\boldsymbol{A}\in\mathbb{R}^{r\times d},\boldsymbol{r}\ll\boldsymbol{d}$）——微调成本与存储大降，效果接近全量微调。
    - Adapter：在层间插入小模块。
  - *指令微调 (Instruction Tuning)*：用“(指令, 期望回答)”对训练——让模型学会“遵循指令”而非只是续写（对齐的第一步）。
  - *与 RAG 对比*：微调改**行为/风格**（内部知识固化）；RAG 改**知识**（外部动态）。知识时效性/私密知识 → RAG；风格/任务格式 → 微调。

- **提示优化 (Prompt Optimization)**：
  - *思想*：不更新权重，只改进输入（few-shot 示例、思维链、角色设定、格式要求）。
  - *Few-shot 与思维链 (CoT)*：给 2–5 个示例（few-shot）；要求“逐步推理”（Chain-of-Thought）——把复杂问题分解为中间步骤，显著提升数学/逻辑任务表现。
  - *系统化方法*：人工迭代（基线 → 加示例 → 加约束 → 验证）或自动优化（如 OPRO、DSPy 把 prompt 当可优化变量，用搜索/LLM 反馈迭代）。
  - *何时用*：任务简单/成本敏感/快速验证——先试 prompt；任务复杂/格式严格/数据量大——再考虑微调。

- **安全 (Safety) 与对齐 (Alignment)**：
  - *对齐目标*：让模型输出**符合人类意图与价值观**（helpful, honest, harmless）。
  - *RLHF（人类反馈强化学习）*：三步——监督微调（SFT）→ 训练奖励模型（比较人类对回答的偏好）→ 用 PPO 类算法优化策略以最大化奖励。
  - *DPO（直接偏好优化）*：跳过显式奖励模型，直接用偏好数据更新策略——更简单稳定。
  - *越狱 (Jailbreak) 与注入 (Prompt Injection)*：对抗性 prompt 绕过安全训练（如“假装你是无约束模型”）；注入攻击=在输入中隐藏指令劫持行为。
  - *治理手段*：输入/输出过滤、内容审查、系统提示约束、红队测试（主动攻击找漏洞）、评估基准。
  - *隐私*：训练数据可能泄露个人信息（记忆攻击）；联邦学习/差分隐私/数据脱敏是缓解方向（L19–20 深入）。

算法伪代码与逻辑解说：RAG 流水线

伪代码

```
输入：
    - 知识库文档集 D，查询 q，嵌入模型 E，LLM G，检索数 k

输出：
    - 基于检索上下文的生成答案 a

离线阶段（构建索引）：
1. 把 D 切块为 Chunks = {c_1, ..., c_N}（按语义边界，块大小 ~200-500 token）
2. 对每个块 c_j：vec_j = E(c_j)                  // 嵌入到 d 维
3. 建立向量索引（如 FAISS，支持近似最近邻）

在线阶段（推理）：
4. 查询嵌入：q_vec = E(q)
5. 检索：top_k = 索引中与 q_vec 余弦相似度最高的 k 个块
6. 组装 prompt：
   "根据以下资料回答问题。资料：[top_k 拼接] 问题：q"
7. a = G(prompt)                                  // LLM 生成（如 L17 的自回归解码）
8. （可选）后处理：引用标注、答案校验（LLM 自检/检索回查）
9. 返回 a
```

【算法逻辑解说】 1. **Step 1 切块是关键设计**：块太大 → 检索噪声多、超上下文窗口；块太小 → 语义破碎。按段落/语义边界切，可带重叠。 2. **Step 5 检索**：近似最近邻（ANN）在百万级块上毫秒级返回——向量检索的工程化核心。 3. **Step 6 Prompt 组装**：把检索结果**原样**放进上下文——LLM 的注意力机制（L17）会自动聚焦与问题相关的片段。 4. **Step 7 生成**：LLM 在“开卷”条件下作答；可加“若资料中无答案，请说明”的指令抑制幻觉。 5. **Step 8 校验闭环**：高级 RAG 增加重排序（rerank 精排 top_k）、生成后引用核查（对每个事实回查资料）——把“可能幻觉”变成“可追溯引用”。 6. **评估指标**：检索质量（Recall@k）与生成质量（答案正确率、引用忠实度）分开评测——定位瓶颈在检索还是生成。

关键要点

1. RAG = 记忆外包：检索（向量相似度）→ 上下文 → 生成；幻觉↓、知识实时更新。
2. 微调 = 行为定制：全量 or LoRA/PEFT；指令微调是“遵循指令”的关键步骤。
3. 提示优化 = 零成本干预：few-shot、思维链、角色约束；先 prompt 后微调。
4. 对齐（RLHF/DPO）把“能力”变成“可控性”；越狱/注入是持续的攻防。
5. 选型矩阵：知识时效/私密 → RAG；风格/任务格式 → 微调；快速验证/低成本 → prompt。

常见误区与注意事项

- **RAG 检索质量差就怪 LLM**：幻觉往往源于“检索不到”而非“生成不好”——先评估检索 Recall@k，再谈生成。
- **微调“教新知识”**：微调学新事实慢且易灾难性遗忘；知识类需求首选 RAG。
- **过度微调导致能力退化**：任务数据太少/太偏 → 通用能力下降（灾难性遗忘）；用 LoRA 等 PEFT 缓解。
- **把 prompt 工程当万能**：复杂推理/严格格式需求下 prompt 收益有限——组合使用（prompt + RAG + 微调）才是实战。

- **忽视注入攻击**：把不可信外部文本拼进 prompt 可能被劫持——隔离不可信内容、输出过滤、权限最小化。
- **安全评估只看基准**：基准通过 ≠ 真实安全（越狱持续演化）——持续红队与监控。

思考题

1. **问题**：RAG 与微调在“知识更新”上的本质区别是什么？
   - **答案**：RAG 的知识在**外部索引**——更新文档库即更新知识，LLM 权重不动；微调把知识写进**权重**——更新需重新训练，且新知识易与旧知识冲突（遗忘）。RAG 适合高频更新/私密数据；微调适合稳定行为模式。
2. **问题**：LoRA 为什么能以少量参数接近全量微调效果？
   - **答案**：研究表明微调时权重更新的**有效秩很低**（本质低维）——$\boldsymbol{W + BA}$ 用低秩矩阵 $\boldsymbol{BA}$（秩 $\boldsymbol{r}$）近似全秩更新 $\boldsymbol{\Delta W}$，参数从 $\boldsymbol{d^2}$ 降到 $\boldsymbol{2dr}$。低秩假设在多数任务上成立，故效果好且可插拔（多个任务 LoRA 可叠加切换）。
3. **问题**：思维链提示为什么能提升推理？它可能失效在何处？
   - **答案**：显式中间步骤把“一步到位”的高难度映射分解为多步低难度映射，且每一步都在上下文可见（注意力可聚焦），减轻单步出错的影响。失效场景：问题超出模型能力上限、提示诱导错误路径、对幻觉敏感的任务（编造的“推理”同样流畅）——所以需要验证/工具结合，而非盲信 CoT 输出。

Lecture 19: Fairness, Algorithmic Bias（公平性与算法偏见）

概述

机器学习系统的决策会**放大社会偏见**：训练数据中的历史歧视、代理变量的偏差、优化目标的不公，都会转化为系统的系统性不公。本讲（与 L20 构成一个模块）先讲**偏见从何而来**与**公平性的数学定义**（群体公平指标及其不可能三角），再讨论**如何测量与缓解**偏见。

核心概念与数学直觉

- **算法偏见的来源**：
  1. **数据偏见 (Data bias)**：训练数据本身反映历史歧视（如招聘数据中男性居多）、采样偏差（某些群体欠代表）、标注者偏见。
  2. **代理变量 (Proxies)**：模型无法直接用敏感属性（种族、性别），但其他特征（邮编、姓氏、消费习惯）与其强相关——偏见“绕道”进入模型。
  3. **目标函数偏见**：优化“整体准确率”会天然偏向多数群体（多数类的错误更少）。
  4. **反馈回路**：系统决策影响未来数据（如偏见招聘 → 未来申请者分布更偏）——偏见自我强化。
- **公平性的数学定义（群体公平）**：设敏感属性 $A \in \{a, b\}$（如性别）、预测 $\hat{Y}$、真实标签 Y。
  - *统计均等 (Demographic Parity)*：预测结果与敏感属性无关：

$$P(\hat{Y} = 1 | A = a) = P(\hat{Y} = 1 | A = b)$$

```
    —各组被“接受”的比例相同。
*   *均等化几率（Equalized Odds)*：错误率与敏感属性无关：
```

$$P(\hat{Y}=1|Y=y,A=a)=P(\hat{Y}=1|Y=y,A=b),\quad y\in\{0,1\}$$

```
    —各组有相同的 TPR（真正例率）与 FPR（假正例率）。
*   *机会均等（Equal Opportunity)*：Equalized Odds 的放松版，只要求 $y=1$（正类）时 TPR 相同—各组'
*   *校准（Calibration)*：预测概率在各组内都准确—$P(Y=1 | \hat{p} = p, A = a) = p$ 对所有组。
*   *不可能三角（Impossibility)*：除平凡情形外，**统计均等、均等化几率、校准三者无法同时满足**（除非 $P(
```

- **偏见缓解的三阶段**：| 阶段 | 方法 | 思想 ||---|---|---|| 预处理 (Pre-processing) | 重采样/重加权、数据去偏 | 修正训练数据的不平衡 || 处理中 (In-processing) | 约束优化（把公平约束加入损失）、对抗去偏 | 训练时强制公平 || 后处理 (Post-processing) | 阈值调整（各组用不同决策阈值） | 不改模型，改决策规则 |
- **测量与评估**：按敏感属性**分层报告**性能（准确率、TPR、FPR、校准误差）；对比各组的指标差距（disparity）；警惕“平均指标掩盖组间差异”。

算法伪代码与逻辑解说：公平性审计

伪代码

```
输入:
    - 模型 M，带敏感属性 A 的评估数据 D（含真实标签 Y），公平指标集合

输出:
    - 分组的性能报告与公平差距

1. 用 M 对 D 预测，得到预测 Ŷ（及概率分数 p̂）
2. 按敏感属性 A 分组: D_a, D_b
3. 对每组 g ∈ {a, b}:
    3.1 准确率，精确率，召回率，F1
    3.2 TPR = P(Ŷ=1 | Y=1, A=g);  FPR = P(Ŷ=1 | Y=0, A=g)
    3.3 接受率 R_g = P(Ŷ=1 | A=g)          // 统计均等检查
    3.4 校准曲线: 预测概率 vs 真实正例率（分桶对比）
4. 计算公平差距:
    4.1 统计均等差距 = |R_a − R_b|
    4.2 均等化几率差距 = |TPR_a − TPR_b| + |FPR_a − FPR_b|
    4.3 机会均等差距 = |TPR_a − TPR_b|
5. 输出报告: 各指标差距 + 组间最差表现组
6. 若差距超阈值 → 进入缓解流程（数据/训练/后处理）
```

【算法逻辑解说】 1. **Step 3 分组评估是前提**：只报“总体准确率 95%”会掩盖“A 组 80%、B 组 99%”的悬殊——公平审计必须**分层报告**。 2. **Step 4 差距度量**：不同指标回答不同问题——接受率（统计均等）关注“结果平等”；TPR/FPR（均等化几率）关注“错误平等”。**选择指标即选择价值立场**。 3. **Step 6 阈值触发缓解**：差距超业务阈值才动刀——过度修正可能损害整体效用（公平–效用权衡）。 4. **注意**：真实标签 Y 本身可能含偏见（如“是否被录用”标签受历史歧视影响）——审计结果的解释需谨慎。

关键要点

1. 偏见来自数据、代理变量、目标函数与反馈回路——不只是“模型问题”。
2. 三大群体公平指标（统计均等 / 均等化几率 / 校准）互不相容——公平没有唯一的数学定义。
3. 缓解可发生在预处理、训练中、后处理三阶段。
4. 公平审计 = 分层报告 + 差距度量；指标选择是伦理决策。
5. 公平性与效用（准确率）常有权衡——需在应用语境中平衡。

常见误区与注意事项

- **“去除敏感属性就公平了”**：代理变量让偏见绕道（邮编≈种族）——需检查特征与敏感属性的相关性。
- **把“公平”当单一指标**：不同公平定义互相冲突（不可能三角）——先明确你关心的不公平类型。
- **忽视组间样本量差异**：小样本组统计噪声大——评估要报告置信区间。
- **只做总体评估**：总体指标掩盖组间差异——务必分层报告。
- **公平修正过头**：为达统计均等强行拉平接受率，可能损害合格者的利益（逆向歧视）——权衡需审慎。

思考题

1. **问题**：为什么“删除种族特征”通常不能消除种族偏见？
   - **答案**：其他特征（邮编、姓名、社会网络、消费习惯）与种族高度相关，是**代理变量**——模型可通过它们“重构”敏感信息。删除显式属性只是移除直接通道，间接通道仍在。对策：检测代理相关性、约束优化、对抗去偏。
2. **问题**：统计均等与均等化几率各适合什么场景？举一例说明它们冲突。
   - **答案**：统计均等适合“结果份额”重要且基础率不同的场景（如信贷获批率）；均等化几率适合“错误代价”重要且需公平容错（如医疗筛查 TPR 公平）。冲突例：若 A、B 两组真实患病率不同，要求 TPR 相同（均等化几率）与要求确诊率相同（统计均等）一般不可兼得——不可能三角的直观体现。
3. **问题**：为什么“整体准确率最高”的模型可能在公平性上很差？
   - **答案**：整体准确率由多数群体主导——优化它等于优先保证多数群体正确，少数群体错误被平均掩盖。例：99% 多数群体 + 1% 少数群体，模型全猜多数类也有 99% 准确率，但少数群体 100% 被误判。公平审计的分层报告正是为了暴露这类“被平均掩盖的不公”。

Lecture 20: Explainability, Privacy（可解释性与隐私）

概述

在公平性之后，本讲讨论让机器学习**可信**的另外两个支柱：**可解释性**（模型为什么这么决策？如何让人理解与审计？）与**隐私**（模型会泄露训练数据吗？如何在保护隐私的同时学习？）。两者都是“AI 治理”的核心议题，与 L19 的公平性共同构成现代 ML 的责任框架。

核心概念与数学直觉

- **可解释性 (Explainability)**：
  - *动机*：高风险决策（医疗、司法、信贷）需要问责与审计；开发者需要调试；用户需要信任。
  - *两类方法*：
    - **固有可解释 (Intrinsically interpretable)**：模型本身透明——线性回归（系数=影响）、决策树（规则路径）、kNN（邻居）。
    - **事后解释 (Post-hoc)**：对黑盒模型（深度网络、GBDT）生成解释。
  - *事后解释代表方法*：
    - **特征重要性 (Feature Importance)**：置换重要性（打乱某特征看性能下降多少）；基于梯度（$\frac{\partial f}{\partial x_j}$ 衡量敏感度）。
    - **LIME**：在预测点附近拟合**局部可解释的线性模型**，用其系数解释局部行为。
    - **SHAP**：基于合作博弈论的 **Shapley 值**——每个特征对预测的**边际贡献**（公平分摊“预测值 − 均值预测”）。Shapley 值满足可加性、对称性等公理，是特征归因的黄金标准：

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} \left[f_S(x_S) - f_{S \setminus \{j\}}(x_{S \setminus \{j\}}) \right]$$

```
        *    $F$：特征全集；$S$：特征子集；$f_S$：只用子集特征的模型输出。
        *    *直觉*：特征 $j$ 的重要性 = 在**所有可能的特征子集组合**下，加入 $j$ 带来的平均边际贡献——公
    - **示例解释**：找训练集中与预测最相似的样本（原型/反事实）。
*   *解释的局限*：近似解释可能误导（“解释的是模型行为，不一定是因果”）；解释的忠实度（faithfulness）难以验
```

- **隐私 (Privacy)**：
  - *威胁模型*：模型在敏感数据（医疗记录、位置、对话）上训练——攻击者可能通过查询**提取训练数据**（记忆攻击：模型“背下”了某些样本）或**推断成员**（判断某样本是否在训练集中）。
  - *差分隐私 (Differential Privacy, DP)*：算法输出对**单个样本的加入/移除**不敏感——数学上保证“无论某人的数据在不在训练集里，输出分布几乎不变”。

$$P(\mathcal{A}(D) \in S) \leq e^{\epsilon} \cdot P(\mathcal{A}(D') \in S)$$

```
    *    $D, D'$：相差一个样本的相邻数据集。
    *    $\epsilon$：隐私预算—越小隐私越强（$\epsilon=0$ 表示输出与数据无关，无实用性）。
    *    *机制*：**加噪**—查询结果加拉普拉斯/高斯噪声，噪声尺度随 $\epsilon$ 与敏感度（单个样本对结果的最
*   *DP-SGD*：训练时对**梯度裁剪 + 加噪**—使整个训练过程满足差分隐私；代价是精度下降与训练变慢。
*   *联邦学习 (Federated Learning)*：数据不离开设备—各客户端本地训练、只上传模型更新，服务器聚合（FedA
*   *其他技术*：数据脱敏/匿名化（k-匿名、l-多样性）、同态加密（在密文上计算）、安全多方计算。
```

- **隐私与公平/可解释的交叉**：隐私保护（加噪、聚合）可能降低少数群体数据质量（公平受损）；可解释性与隐私冲突（解释需要访问模型细节）。

算法伪代码与逻辑解说：DP-SGD（差分隐私随机梯度下降）

伪代码

```
输入：
    - 训练数据 D，损失函数 L，学习率 alpha，隐私预算 ε，噪声尺度 σ，裁剪阈值 C

输出：
    - 满足 (ε, δ)-差分隐私的模型参数 θ

1. 初始化 θ
2. 对每个 epoch：
    2.1 对每个 mini-batch B：
        2.2 对每个样本 i ∈ B：
            g_i = ∇_θ L(θ; x_i, y_i)                    // 逐样本梯度
            g_i = g_i * min(1, C / ||g_i||_2)           // 梯度裁剪：范数限制在 C 内
        2.3 g_bar = (1/|B|) * Σ_i g_i                   // 平均
        2.4 g_tilde = g_bar + N(0, σ²C²)                // 高斯噪声（敏感度=2C/|B| 量级）
        2.5 θ = θ - alpha * g_tilde                     // 更新
3. 返回 θ
```

【算法逻辑解说】 1. **Step 2.2 逐样本梯度 + 裁剪**：DP 要求“单个样本影响有界”——裁剪把每个样本的梯度范数限制在 C，从而**敏感度有界**（这是隐私证明的前提）。 2. **Step 2.4 加噪**：噪声尺度 σ 与隐私预算 ϵ 挂钩（ϵ 越小噪声越大）。噪声**掩盖单个样本的贡献**——攻击者无法从更新中分辨“多了一个你”。 3. **代价**：加噪扰动梯度 → 收敛更慢、最终精度更低——**隐私-效用权衡**。隐私预算 ϵ 累积：多轮训练消耗预算，需用隐私会计（moments accountant）追踪总消耗。 4. **应用**：苹果、谷歌的联邦统计、OpenAI 等机构发布模型时的隐私评估——DP 是“可证明的隐私”标准工具。

关键要点

1. 可解释性：固有（线性/树）vs 事后（LIME/SHAP/重要性）；SHAP 基于 Shapley 值公平分摊特征贡献。
2. 解释是**近似**——忠实度难验证，勿把解释当因果。
3. 隐私威胁：成员推断、训练数据提取（记忆）。
4. 差分隐私 = 输出对单样本不敏感（ϵ 预算控制噪声量）；DP-SGD = 裁剪 + 加噪的训练范式。
5. 联邦学习是分布式范式，**不等于隐私**——需与 DP/加密结合。

常见误区与注意事项

- **把特征重要性当因果**：SHAP/LIME 描述“模型如何用特征”，不证明“特征导致结果”——相关性≠因果。
- **解释局部 vs 全局混淆**：LIME/SHAP 是**局部**解释（针对单个预测）；全局解释（整个模型行为）需要不同方法（如代理模型）。
- **以为删除 PII 就安全**：去标识化可被重识别（与其他数据交叉）；差分隐私提供的是**可证明**的保障而非“感觉安全”。
- **联邦学习=隐私**：FedAvg 的梯度更新可泄漏数据分布信息——必须配 DP/加密才是隐私保护。

- **隐私预算无限使用**：ϵ 会累积——反复发布查询/训练耗尽预算后保障失效。
- **解释与隐私互斥时不做权衡**：两者目标冲突（解释暴露细节 vs 隐私隐藏细节）——按风险场景取舍。

思考题

1. **问题**：Shapley 值为什么被认为是“公平”的特征归因？它与简单“删除特征看性能”有何区别？
   - **答案**：Shapley 值对**所有特征子集顺序**的边际贡献取平均——满足对称性、可加性、哑变量（无关特征贡献 0）、效率（贡献之和 = 预测−基线）等公理；“删除特征”只测单一路径，且受特征相关性与模型重训影响，结果不稳定、不满足公理。Shapley 值是对“公平分摊”的博弈论解。
2. **问题**：为什么说“联邦学习不提供隐私保证”？
   - **答案**：联邦学习解决的是**数据不出本地**的分布式训练问题，但上传的梯度/模型更新**编码了本地数据信息**——攻击者可用梯度反演、成员推断恢复数据或判断某样本是否参与训练。只有在更新上施加差分隐私噪声（DP–FedAvg）或加密协议，才构成隐私保护。
3. **问题**：差分隐私中 ϵ 越小越好吗？实际部署如何取舍？
   - **答案**：ϵ 越小隐私越强但噪声越大、效用越低（$\epsilon = 0$ 输出与数据无关、无实用价值）。实践按风险定预算：高敏感场景（医疗）用 $\epsilon \leq 1$ 量级；低敏感统计用 $\epsilon \in [1, 10]$。同时用隐私会计精确追踪累积消耗，并考虑“隐私–效用”的帕累托前沿——在可接受效用下选最小 ϵ。

---

6. 核心数学与算法速查表（Quick Reference）

6.1 监督学习：核心公式

模型	假设/损失	关键公式
线性回归	高斯噪声 + MLE	$h_\theta(x) = \theta^T x$；$J(\theta) = \frac{1}{2m}\sum_i (h_\theta(x^{(i)}) - y^{(i)})^2$；$\theta = (X^T X)^{-1} X^T y$
梯度下降	—	$\theta_j := \theta_j - \alpha \frac{1}{m}\sum_i (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$（批量）；单样本版为 SGD
局部加权回归	加权 MSE	$J(\theta) = \frac{1}{2}\sum_i w^{(i)} (y^{(i)} - \theta^T x^{(i)})^2$，$w^{(i)} = \exp(-\frac{(x^{(i)} - x)^2}{2\tau^2})$
逻辑回归	Bernoulli + MLE	$h_\theta(x) = \frac{1}{1+e^{-\theta^T x}}$；$\ell = \sum_i [y^{(i)} \log h + (1 - y^{(i)}) \log(1 - h)]$； $\theta_j := \theta_j + \alpha \sum_i (y^{(i)} - h_\theta(x^{(i)})) x_j^{(i)}$
牛顿法	二阶优化	$\theta := \theta - H^{-1} \nabla_\theta \ell(\theta)$，$H_{jk} = \partial^2 \ell / \partial\theta_j \partial\theta_k$
指数族/GLM	$p(y;\eta) = b(y) e^{\eta^T T(y) - a(\eta)}$	$h_\theta(x) = E[T(y)
Softmax 回归	多项分布	$h_\theta(x) = \frac{1}{\sum_j e^{\theta_j^T x}} [e^{\theta_1^T x}, \ldots, e^{\theta_k^T x}]^T$

模型	假设/损失	关键公式
GDA	高斯 + 贝叶斯	$\phi, \mu_0, \mu_1, \Sigma$ 由 MLE 估计；共享 Σ ⇒ 线性决策边界
朴素贝叶斯	条件独立	$P(y
岭回归 (L_2)	高斯先验/MAP	$\theta = (X^T X + \lambda I)^{-1} X^T y$
Lasso (L_1)	拉普拉斯先验	$\min \frac{1}{2m}\|X\theta - y\|^2 + \lambda\|\theta\|_1$ ⇒ 稀疏解（特征选择）
偏差–方差分解	—	$\mathbb{E}[(y - \hat{f})^2] = \sigma^2 + \text{Bias}^2 + \text{Var}$

6.2 SVM 与核方法

概念	公式
核	$K(x, z) = \langle \phi(x), \phi(z) \rangle$；高斯核 $K = \exp(-\frac{\|x-z\|^2}{2\sigma^2})$
SVM 原始问题	$\min_{w,b} \frac{1}{2}\|w\|^2$ s.t. $y^{(i)}(w^T x^{(i)} + b) \geq 1$
软间隔	$\min \frac{1}{2}\|w\|^2 + C\sum_i \xi_i$ s.t. $y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i,\ \xi_i \geq 0$
对偶决策	$h(x) = \text{sign}(\sum_{i \in SV} \alpha_i y^{(i)} K(x^{(i)}, x) + b)$

6.3 无监督学习

算法	核心步骤/公式
K–Means	指派 $c^{(i)} = \arg\min_j \|x^{(i)} - \mu_j\|^2$；更新 $\mu_j = \frac{1}{\|S_j\|}\sum_{i \in S_j} x^{(i)}$；最小化 $J = \sum_i \|x^{(i)} - \mu_{c^{(i)}}\|^2$
GMM	$p(x) = \sum_j \phi_j \mathcal{N}(x; \mu_j, \Sigma_j)$；E 步 $w_j^{(i)} \propto \phi_j \mathcal{N}(x^{(i)}; \mu_j, \Sigma_j)$；M 步加权 MLE
EM	E 步：$Q_i(z)=P(z
PCA	$\Sigma = \frac{1}{m} X^T X$；特征分解取前 k 特征向量 U_k；$z = U_k^T \tilde{x}$
ICA	$x = As$，找 $W = A^{-1}$ 使 Wx 分量独立（最大化非高斯性）

6.4 决策树与 Boosting

算法	核心公式
熵/信息增益	$H(S) = -\sum_c p_c \log_2 p_c$；$\text{Gain}=H(S)-\sum_v\frac{
Gini	$\text{Gini}(S) = 1 - \sum_c p_c^2$
AdaBoost	$\epsilon_t = \sum_{i: h_t(x^{(i)}) \neq y^{(i)}} D^{(i)}$；$\alpha_t = \frac{1}{2}\ln\frac{1-\epsilon_t}{\epsilon_t}$；$D^{(i)} \propto D^{(i)} e^{-\alpha_t y^{(i)} h_t(x^{(i)})}$；$H(x) = \text{sign}(\sum_t \alpha_t h_t(x))$

6.5 神经网络

概念	公式
前向传播	$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$；$a^{[l]} = g^{[l]}(z^{[l]})$
反向传播	$\delta^{[L]} = \partial J / \partial z^{[L]}$（Softmax+CE: $a^{[L]} - y$）；$\delta^{[l]} = (W^{[l+1]T} \delta^{[l+1]}) \odot g'(z^{[l]})$；$\partial J / \partial W^{[l]} = \delta^{[l]} a^{[l-1]T}$
激活导数	Sigmoid: $g(1-g)$；tanh: $1-g^2$；ReLU: $\mathbf{1}\{z>0\}$
梯度下降	$\theta := \theta - \alpha \nabla_\theta J$（Adam 为自适应步长变体）

6.6 强化学习

概念	公式
价值函数	$V^\pi(s)=\mathbb{E}[\sum_t\gamma^t R(s_t)
贝尔曼最优	$V^*(s) = R(s) + \gamma \max_a \sum_{s'} P_{sa}(s') V^*(s')$
值迭代	反复应用上式直至收敛；$\pi^*(s) = \arg\max_a \sum_{s'} P_{sa}(s') V^*(s')$
策略迭代	评估 V^π（解线性方程）→ 改进 $\pi'(s) = \arg\max_a \sum_{s'} P_{sa}(s') V^\pi(s')$
Q-learning	$Q(s,a) \mathrel{+}= \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$（无模型，off-policy）
REINFORCE	$\theta:=\theta+\alpha\sum_t R_t\nabla_\theta\log\pi_\theta(a_t

6.7 现代主题（LLM / 公平 / 隐私）

概念	公式/要点
语言建模	$\mathcal{L}=-\frac1T\sum_t\log P_\theta(x_t
注意力	$\mathbf{Attn}(Q,K,V) = \mathbf{softmax}(\frac{QK^T}{\sqrt{d_k}})V$
余弦相似度（RAG 检索）	$\cos(u,v) = \frac{u \cdot v}{\|u\|\|v\|}$
LoRA	$W' = W + BA$，$B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times d}$，$r \ll d$
公平指标	统计均等：$P(\hat Y=1
差分隐私	$P(\mathcal{A}(D) \in S) \leq e^\epsilon P(\mathcal{A}(D') \in S)$；机制：裁剪梯度 + 加噪（DP-SGD）

---

7. 参考资料与链接

课程官方（公开部分） – 课程主页：https://cs229.stanford.edu/ – Syllabus（Fall 2021，含完整日程与讲义链接）：https://cs229.stanford.edu/syllabus-fall2021.html – Syllabus（Fall 2020 / Spring 2021）：

https://cs229.stanford.edu/syllabus-fall2020.html – Course Logistics & FAQ（Summer 2026，公开 Google Doc）：https://docs.google.com/document/d/1PbQxBQTpp4K5hTzOJB9r8etKgsAuNPWs/

公开讲义 PDF（本次笔记的数学内容主要依据） – cs229-notes1（监督学习/线性回归/逻辑回归/GLM）：http://cs229.stanford.edu/notes2021fall/cs229-notes1.pdf – cs229-notes2（生成学习/GDA/朴素贝叶斯）：http://cs229.stanford.edu/notes2021fall/cs229-notes2.pdf – cs229-notes3（核方法/SVM）：http://cs229.stanford.edu/notes2021fall/cs229-notes3.pdf – cs229-notes5（正则化与模型选择）：http://cs229.stanford.edu/notes2021fall/cs229-notes5.pdf – cs229-notes7a（K-Means）/ notes7b（GMM 与 EM）：http://cs229.stanford.edu/notes2021fall/cs229-notes7a.pdf – cs229-notes8（EM 算法）、notes9（因子分析）、notes10（PCA）、notes11（ICA）、notes12（强化学习） – deep_learning_notes（神经网络与反向传播）：http://cs229.stanford.edu/notes2021fall/deep_learning_notes.pdf – ML-advice（调试机器学习系统）：http://cs229.stanford.edu/materials/ML-advice.pdf – 决策树/Boosting/GMM/EM 幻灯片（Fall 2021）：http://cs229.stanford.edu/notes2021fall/lecture11-decision-trees.pdf 等

受限资源（需 Stanford 账号，本笔记未使用）：当季 Syllabus 与讲义（Google Drive）、Canvas（录播/日历）、Ed 论坛、Problem Sets、期中/期末试卷、最终项目。

> **免责声明**：本笔记为学习用途整理，基于公开 Syllabus、公开讲义与通用机器学习知识撰写；数学符号以 CS229 讲义为准。如用于正式学习，请以课程官方当季材料为准。

—— 完 ——